Laboratory: GNSS Positioning with the SPP Method

Week 9 — Collect, validate, compete on a leaderboard

AAE4203 – Guidance and Navigation

Dr Weisong Wen Assistant Professor

- **Lab** Trustworthy AI and Autonomous Systems Laboratory (TAS Lab)
- **Dept** Aeronautical and Aviation Engineering, PolyU
- **Time** Week 9, Semester 1, 2026/27 · 30 October 2026

7	57	20	1
sections	slides	groups	leaderboard

AAE4203 · Semester 1, 2026–27 · PolyU

THE BIG PICTURE



By the end of the lab you can:

- collect raw GNSS data with u-center and a ZED-F9P (15–20 min per group)
- process and validate your own data with RTKLIB and your Python SPP code
- improve accuracy with AI-assisted coding: multi-constellation, weighting, outlier rejection
- submit a benchmark trajectory and compete on the leaderboard

Contents

Plan, field collection, processing and validation, the benchmark challenge, the report and a worked example

01

Lab Plan & Groups

Session plan, the three datasets and the groups

02

Field Data Collection

Software, receiver set-up and u-center logging

03

Processing & Own-Data Validation

RTKLIB, CSV conversion, your SPP code and validation

04

Benchmark Challenge & Leaderboard

Benchmark, accuracy ideas, AI rules, ranking, upload

05

Report & Checklist

Report contents and checklist

06

Example Dataset: Block X

The warm-up dataset, worked through step by step

07

Resources

Tutorial, manuals and references

Lab Introduction



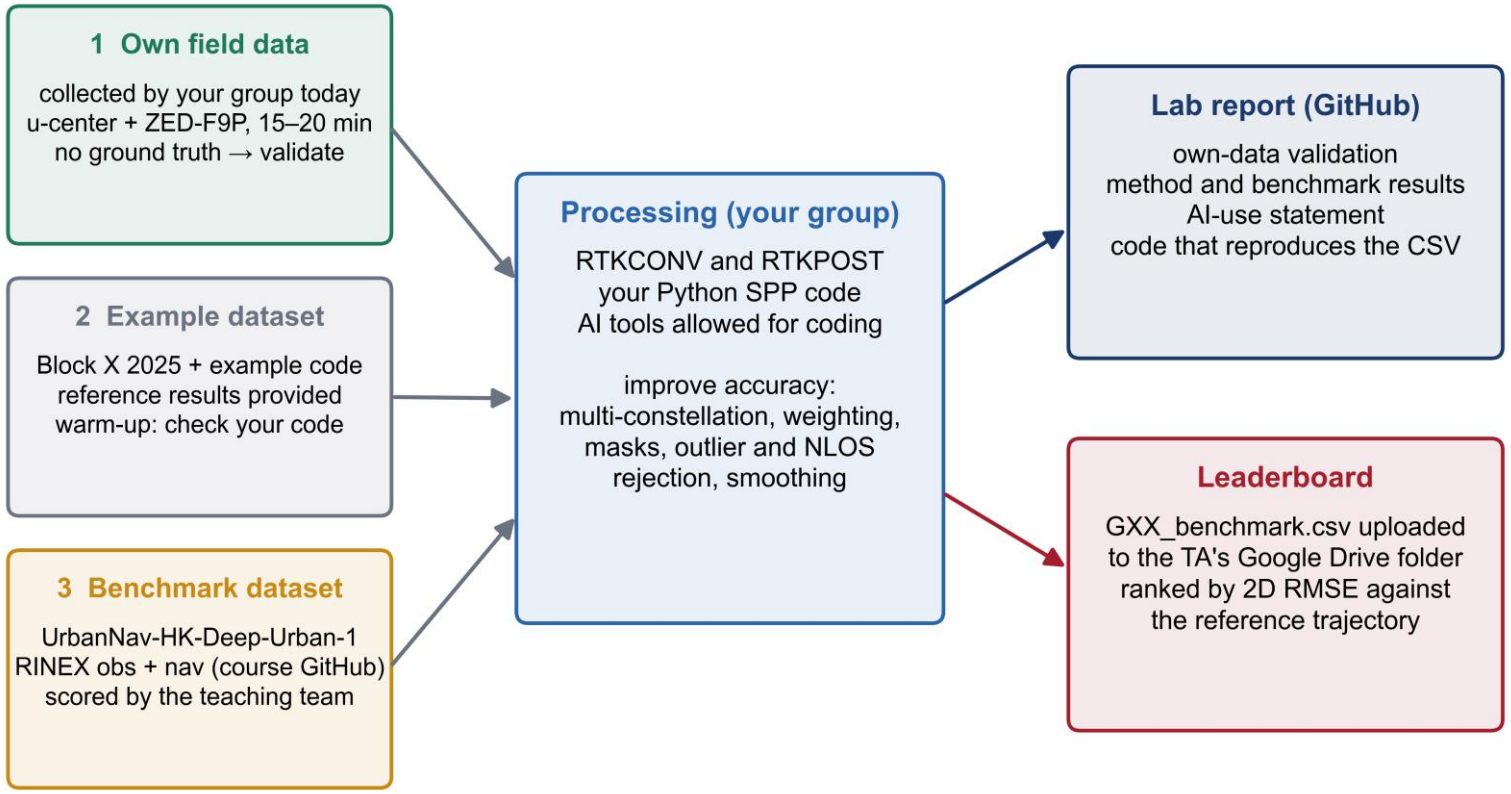


This lab focuses on the practical application of GNSS, from data collection with a receiver to data processing using both professional software and custom Python scripts. Upon successful completion of this lab, you will be able to:

- ***Objective 1***: Operate GNSS data collection software, especially u-center for u-blox receivers.
- ***Objective 2***: Process raw GNSS data using the open-source RTKLIB software suite to estimate position and velocity.
- ***Objective 3***: Develop Python scripts to process GNSS data for position estimation.
- ***Objective 4***: Analyze GNSS data, including the evaluation of positioning error for a static dataset collected in a challenging urban environment, using Python.

The Lab at a Glance (2026/27)

Three datasets, one processing chain, two outputs



What is new this year

Own data — every group collects and validates its own measurements

Benchmark — urban dataset from UrbanNav

Leaderboard — upload a CSV trajectory; groups are ranked by accuracy

AI tools — allowed for coding, with rules

10 kits, 2 batches — same steps; batch 2 starts with the example

Collect → validate your own data → compete on the benchmark

01

Lab Plan & Groups

Section 1 of 7

- Session plan: 10 kits, 2 batches
- Three datasets, three questions
- Groups G01–G20

1 **Lab Plan & Groups**

2 Field Data Collection

3 Processing & Own-Data Validation

4 Benchmark Challenge & Leaderboard

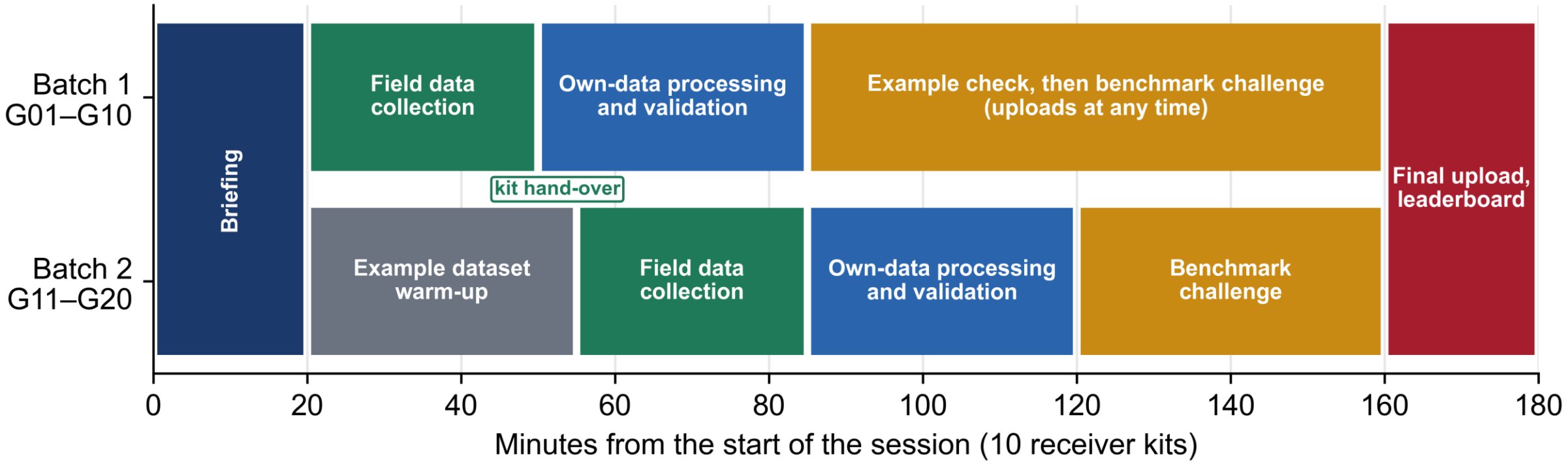
5 Report & Checklist

6 Example Dataset: Block X

7 Resources

Session Plan: 10 Kits, 2 Batches

3-hour session on 30 October 2026 · every group: collect → validate own data → benchmark



Final ranking uses the files at the deadline, so both batches have the same total time

Three Datasets, Three Questions

Each dataset answers a different question about your work

1 Your own data

Can we collect and process real data?

What: static, 15–20 min, near Block X

Truth: none → validate by comparison

When: batch 1 at 0:20, batch 2 at 0:55

Output: own-data part of the report

2 Example dataset

Does our code give the right answer?

What: Block X, 2025, UBX/RINEX/CSV + example code

Truth: reference results provided

When: before the lab (batch 2 also at 0:20) and after every code change

Output: confidence that the code works

3 Benchmark dataset

How accurate is our method?

What: UrbanNav-HK-Deep-Urban-1 (Hong Kong), RINEX obs + nav

Truth: reference trajectory kept by the teaching team

When: after validating your own data

Output: leaderboard rank

Only the benchmark is ranked — one reference trajectory for all groups makes it fair

Grouping (2026/27)

Same groups as the group presentation · one kit, one dataset and one report per group

78 students → 20 groups → 18 × 4 and 2 × 3 → 10 kits · 2 batches

Each group	Suggested roles
Borrows one kit: u-blox ZED-F9P + antenna (bring a laptop) Collects one dataset near Block X Submits one benchmark CSV, one own-data CSV and one report	1 Hardware set-up and logging (u-center) 2 Conversion and RTKLIB processing 3 SPP coding and accuracy improvement 4 Analysis, report and GitHub repository

Every member must be able to explain the whole workflow and the submitted code

Group List: G01–G10

Member names and student numbers are on Blackboard (removed from the public version)

Group	Batch	Kit	Group size
G01	**1**	K01	4 members
G02	**1**	K02	4 members
G03	**1**	K03	4 members
G04	**1**	K04	3 members
G05	**1**	K05	4 members
G06	**1**	K06	4 members
G07	**1**	K07	4 members
G08	**1**	K08	4 members
G09	**1**	K09	3 members
G10	**1**	K10	4 members

Batch 1 (G01–G10): collect at 0:20–0:50, then hand kit Kn to the partner group G(n+10)

Group List: G11–G20

Member names and student numbers are on Blackboard (removed from the public version)

Group	Batch	Kit	Group size
G11	**2**	K01	4 members
G12	**2**	K02	4 members
G13	**2**	K03	4 members
G14	**2**	K04	4 members
G15	**2**	K05	4 members
G16	**2**	K06	4 members
G17	**2**	K07	4 members
G18	**2**	K08	4 members
G19	**2**	K09	4 members
G20	**2**	K10	4 members

Batch 2 (G11–G20): collect at 0:55–1:25 with the kit of the partner group G(n−10), then return it

02

Field Data Collection

Section 2 of 7

- Step-by-step overview
- Software and code
- Steps 1–3: connect and record

1 Lab Plan & Groups

2 **Field Data Collection**

3 Processing & Own-Data Validation

4 Benchmark Challenge & Leaderboard

5 Report & Checklist

6 Example Dataset: Block X

7 Resources

Field Data Collection: Step by Step

30-minute slot: batch 1 at 0:20, batch 2 at 0:55 · the next slides show each step

Step	What to do	Time
1	Pick up the kit; connect antenna → ZED-F9P → USB → laptop	5 min
2	Place the antenna at your site near Block X, clear view of the sky	3 min
3	u-center: choose the COM port, check satellites and C/N_0	2 min
4	Record raw data to a .ubx file — keep the antenna still	15–20 min
5	Stop, copy the file, take photos; hand the kit to your partner group	3 min

Good practice

- Note the start and end time
- Do not stand or walk next to the antenna
- Check that the .ubx file keeps growing
- Keep the laptop on mains or a full battery
- Back up the .ubx file twice
- Record weather and surroundings

A still antenna and a complete .ubx file are all you need for the rest of the lab

Software Requirements (1/2)

Software Requirements

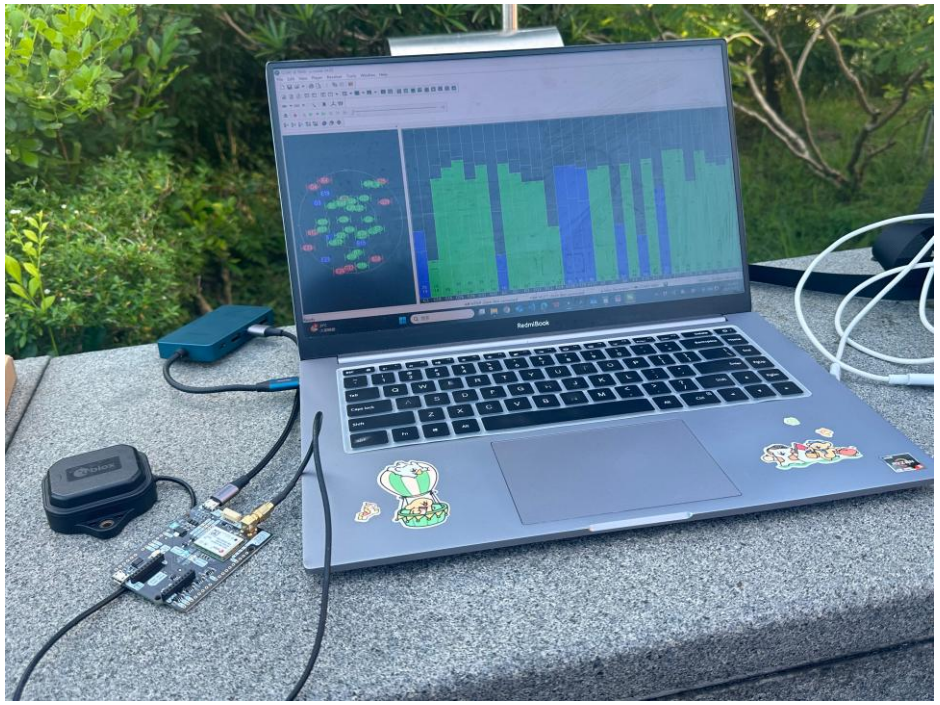
- [u-center](#) for data collection
- [MATLAB](#) with the following toolboxes:
  - Mapping Toolbox
- [Python 3.7+](#) with the following packages:
  - numpy
  - matplotlib
  - pandas
  - pyubx2
  - cartopy (optional, for enhanced map visualization)
- [RTKLIB 2.5](#) (RTKconv and RTKPOST utilities)
- [Google Earth](#) for .kml trajectory visualization



Software Requirements (2/2)

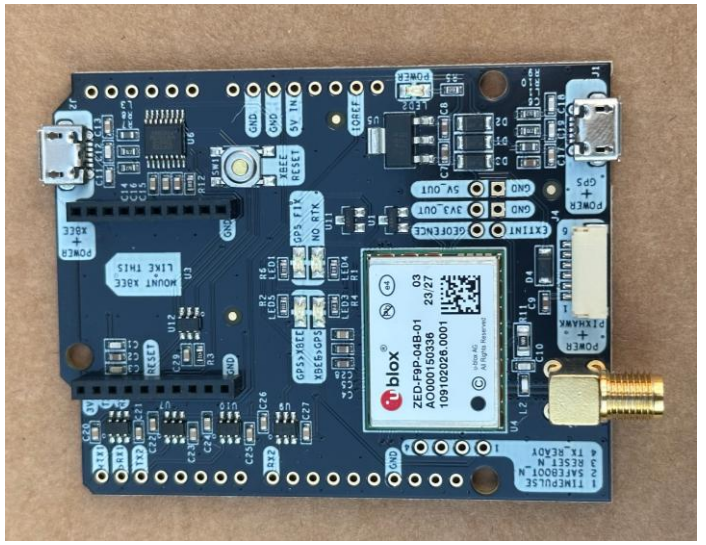
Category	Component	Details
Code	rinex2csv.m	MATLAB script to convert RINEX observation/navigation files into CSV; depends on matRTKLIB.
Code	pyubx2_csv_converter_gui.py	GUI tool to convert UBX/NMEA logs to CSV; depends on pyubx2.
Code	WLSE_spp.py	Reference Python script for Weighted Least Squares Single-Point Positioning ***(provided with the example dataset)***.
Software	u-center	Data collection from u-blox receivers.
Software	MATLAB	Requires Mapping Toolbox.
Software	Python 3.7+	Packages: numpy; matplotlib; pandas; pyubx2; cartopy (optional, for enhanced map visualization).
Software	RTKLIB 2.5	Use RTKconv and RTKPOST utilities for RINEX conversion and post-processing.
Software	Google Earth	Visualize trajectories in .kml format.

Step 1 · Hardware Set-up

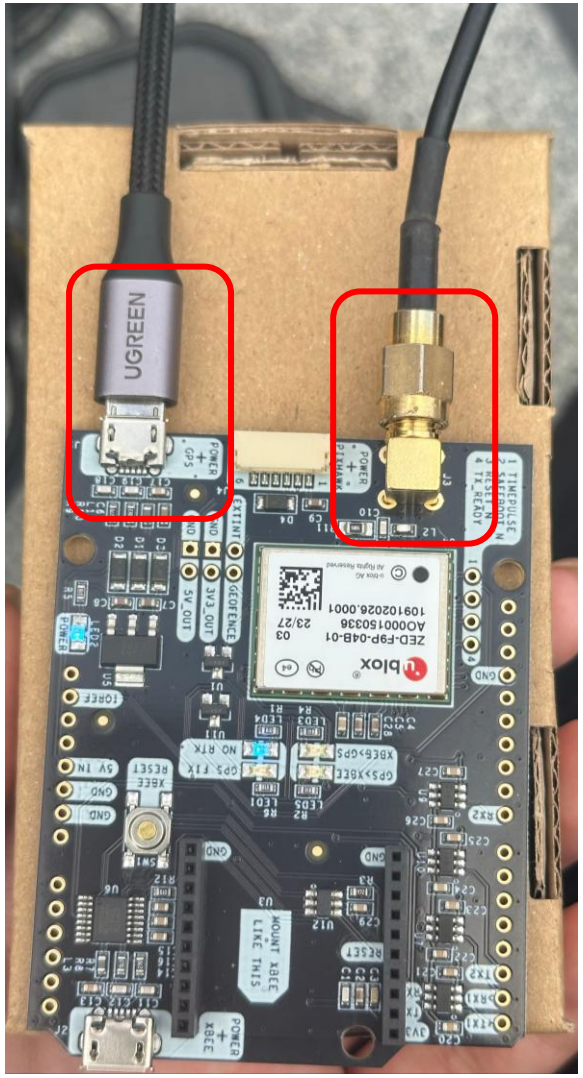


The equipment used for data collection:

- One laptop
- U-blox ZED-F9P + Antenna / U-blox boards



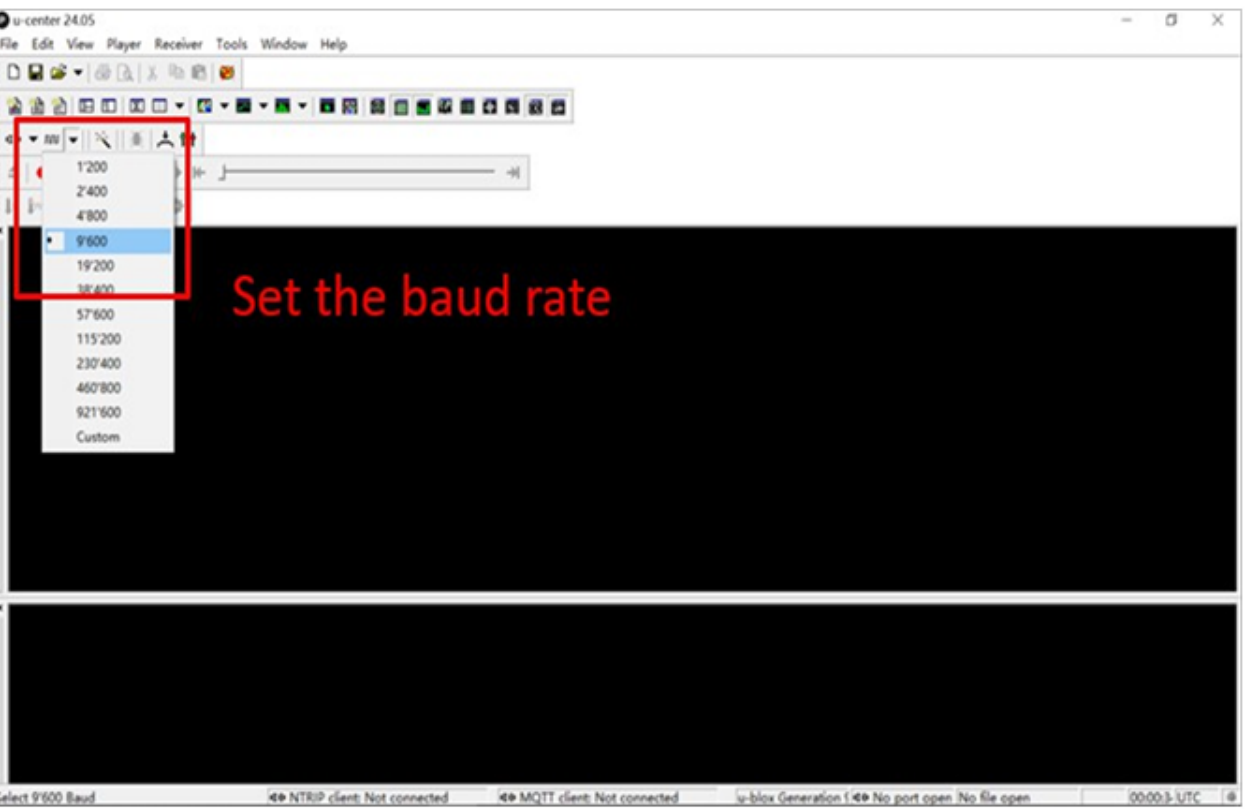


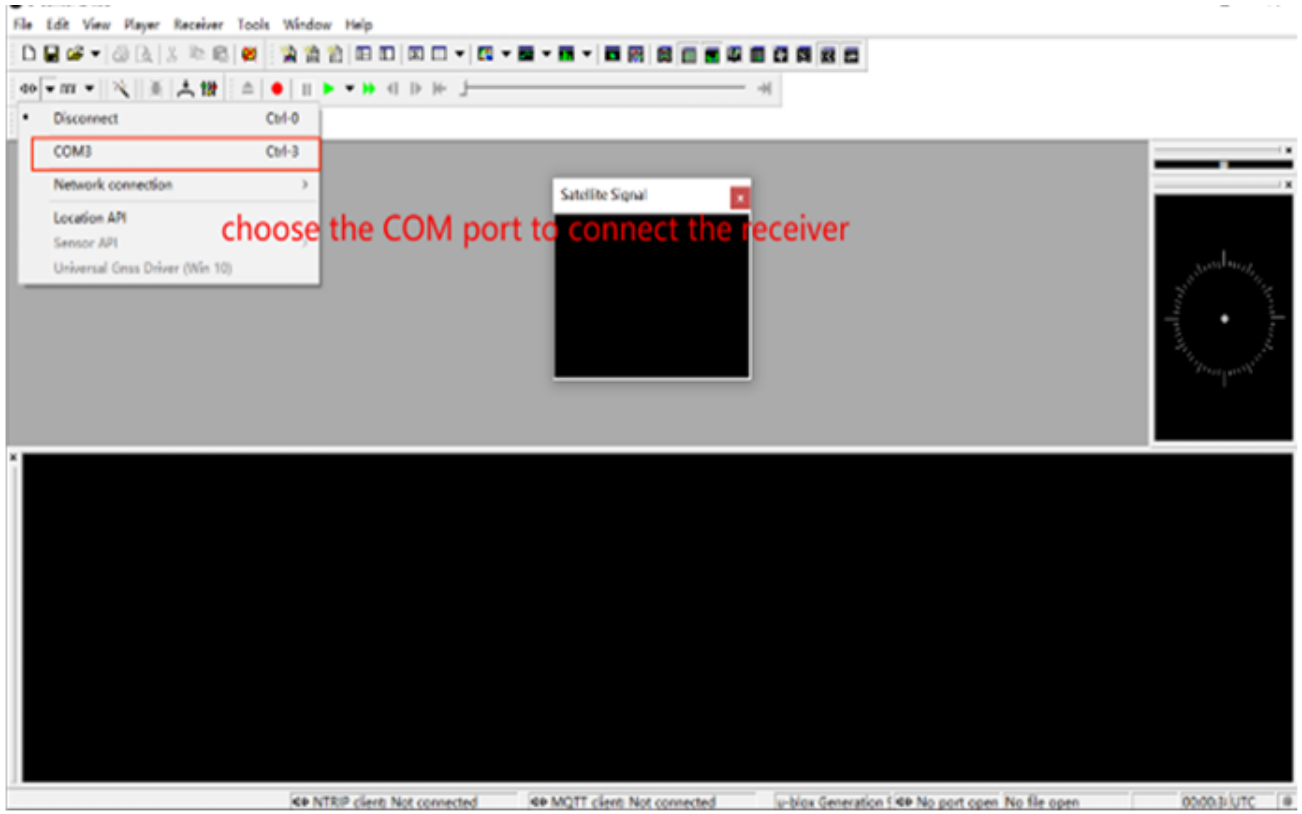


Device connection details

Step 2 · Connect the Receiver in u-center

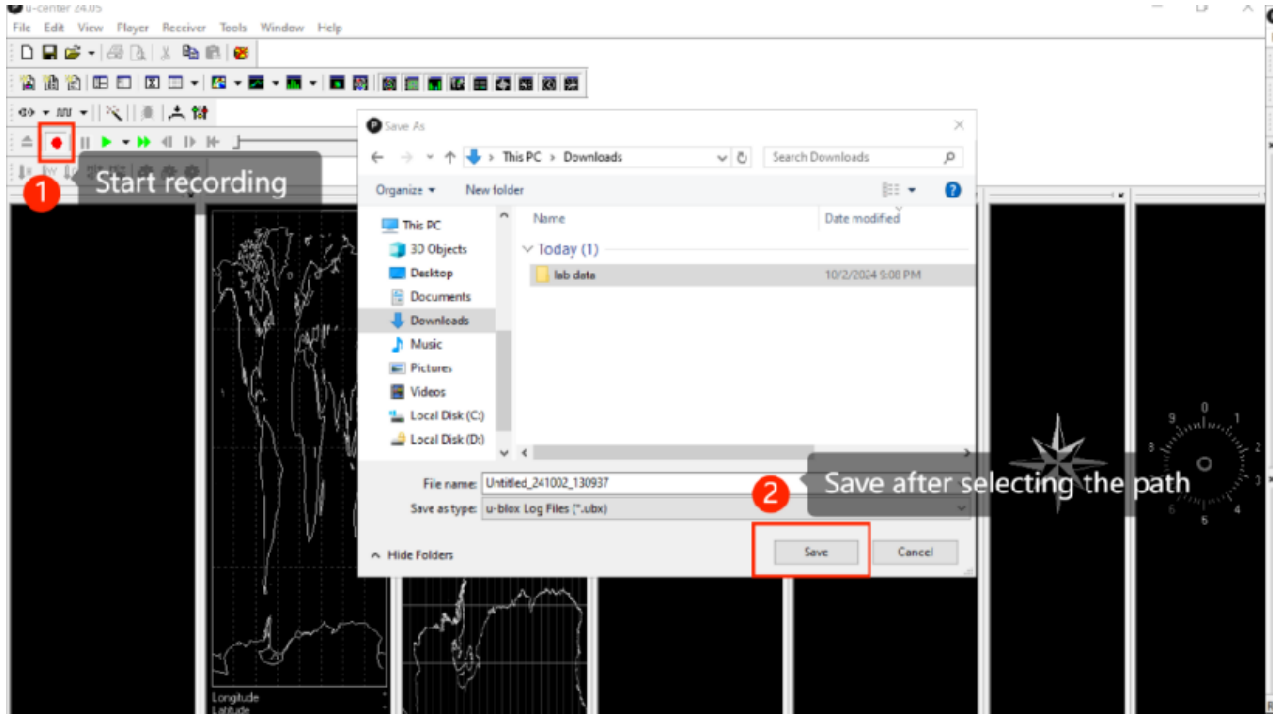
Connect Receiver

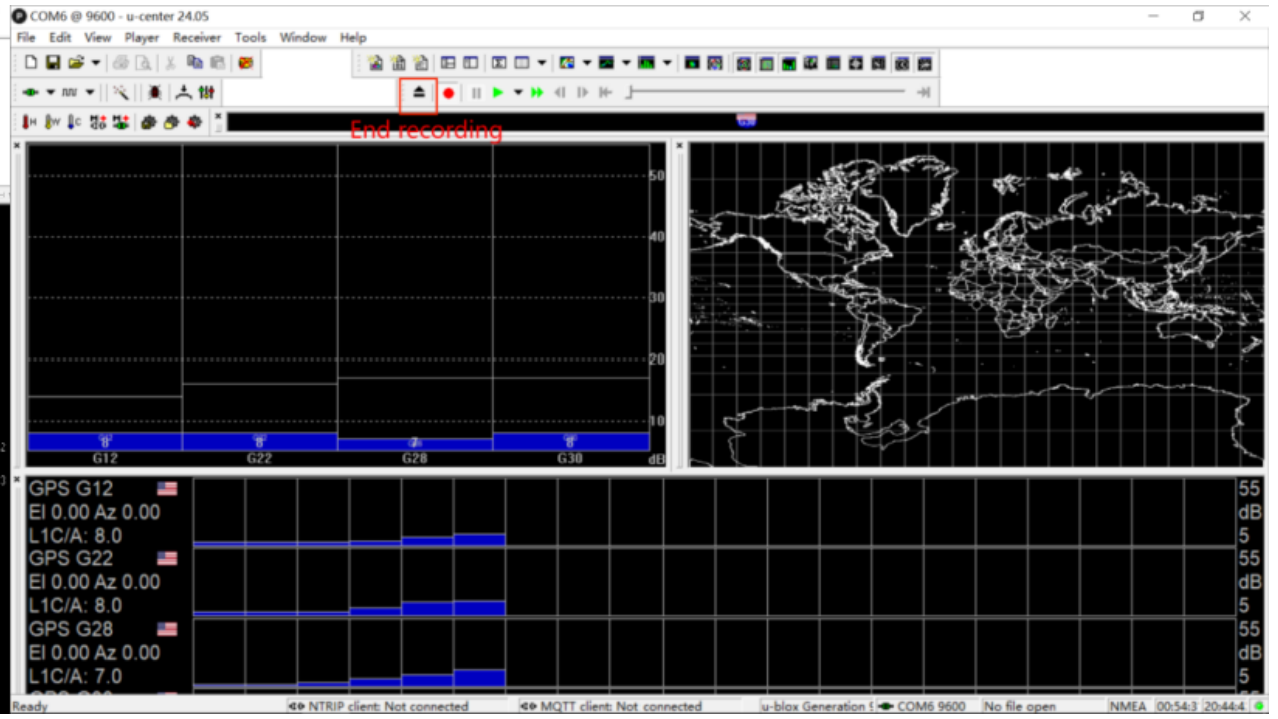




Step 3 · Record Raw Data (UBX)

Data Collection





03

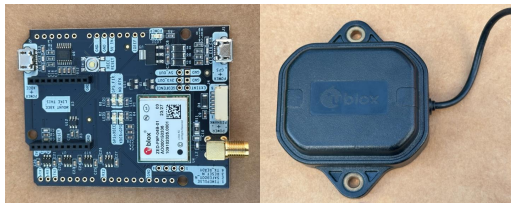
Processing & Own-Data Validation

Section 3 of 7

- Processing pipeline
- Steps 4–8: RTKLIB and CSV files
- Step 9: your SPP code
- Validation without ground truth

1 Lab Plan & Groups
2 Field Data Collection
3 Processing & Own-Data Validation
4 Benchmark Challenge & Leaderboard
5 Report & Checklist
6 Example Dataset: Block X
7 Resources

Processing Pipeline: Files and Tools





GNSS data collection with u-blox

RTKCONV

.obs .nav

rinex2csv.m

- pseudoranges_meas.csv
- satellite_clock_bias.csv
- ionospheric_delay.csv
- tropospheric_delay.csv
- satellite_positions.csv

WLSE_spp.py
(Coding by yourself)

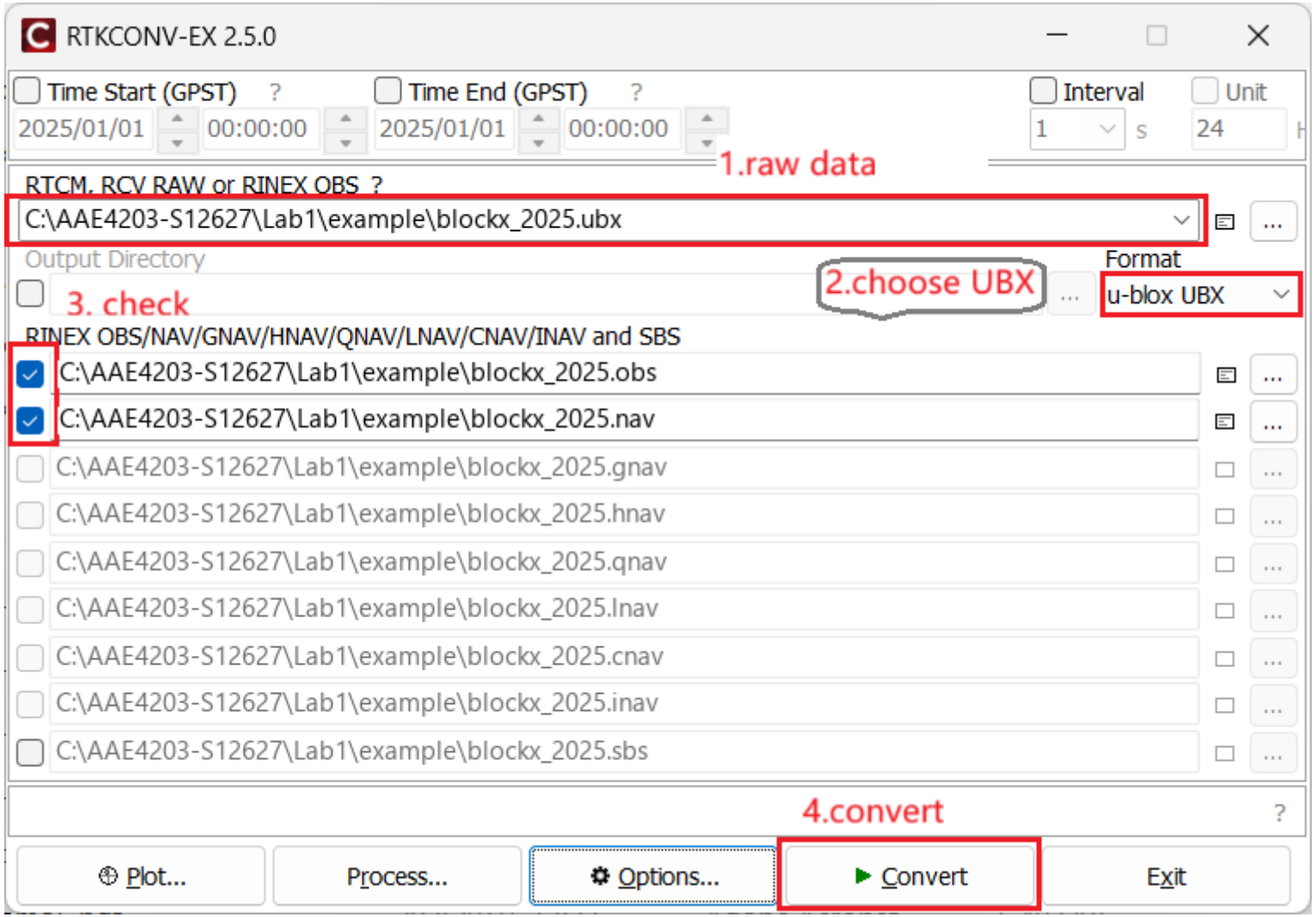
pyubx2_csv_converter_gui.py

NAV-HPPOSECEF.csv
(epoch unit:ms) (position unit: cm)

Receiver reference

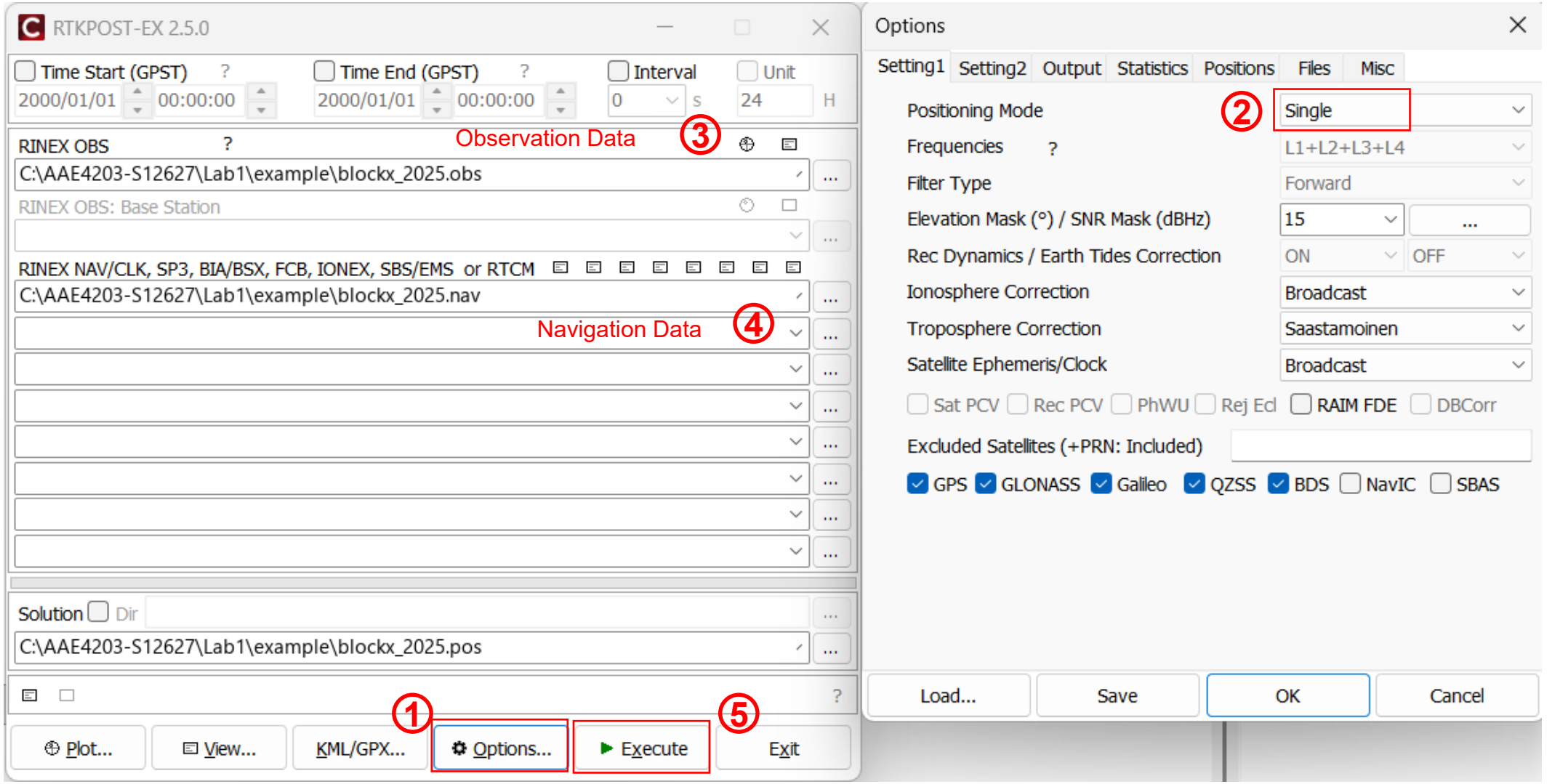
Step 4 · RTKCONV: UBX to RINEX

RTKCONV: UBX → .obs .nav



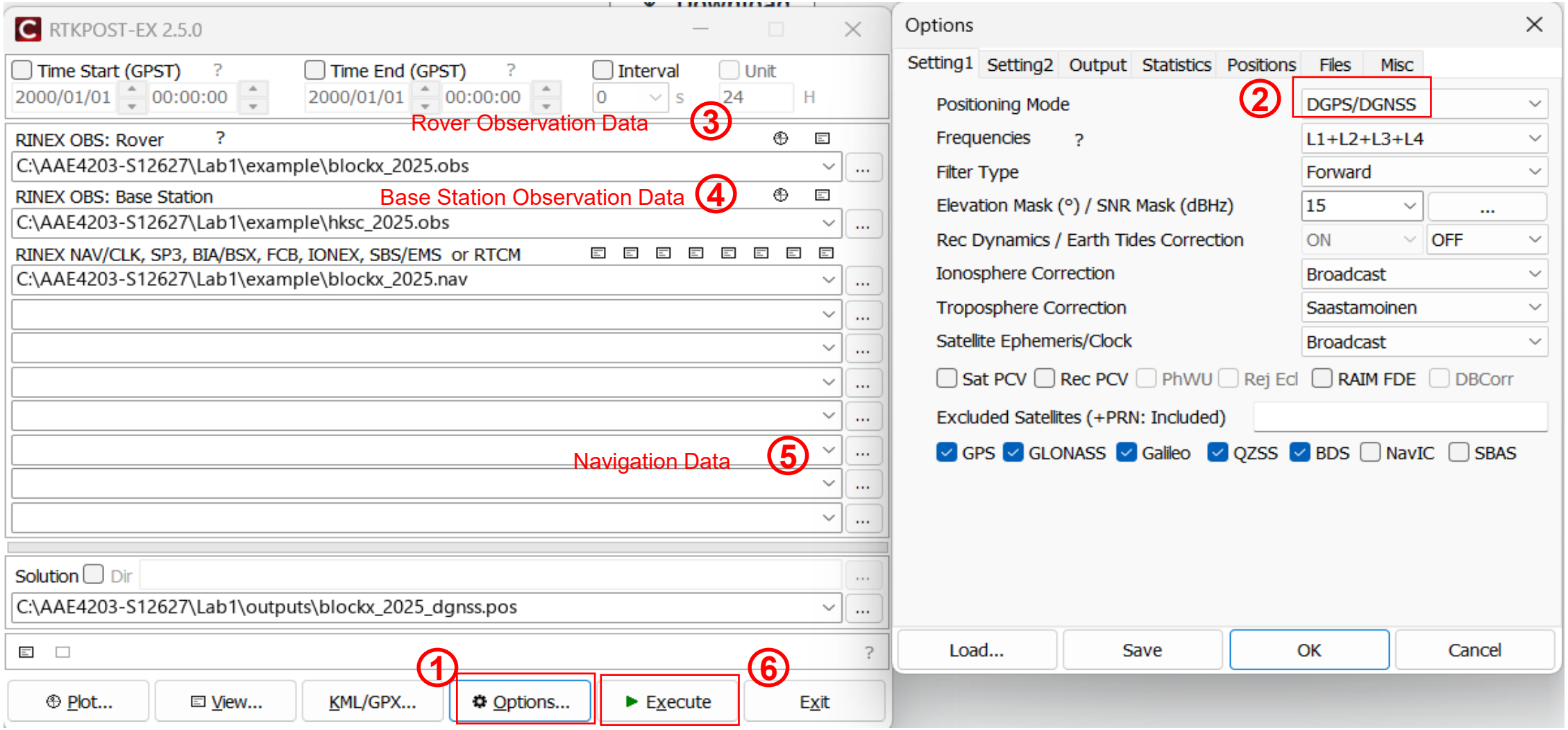
Step 5 · RTKPOST: SPP Solution

RTKPOST: SPP



Step 6 · RTKPOST: DGNSS with HK SatRef

RTKPOST: DGNSS (HK SatRef RINEX; example: example/hksc_2025.obs)



Step 7 · rinex2csv: RINEX to CSV Inputs

matRTKLIB

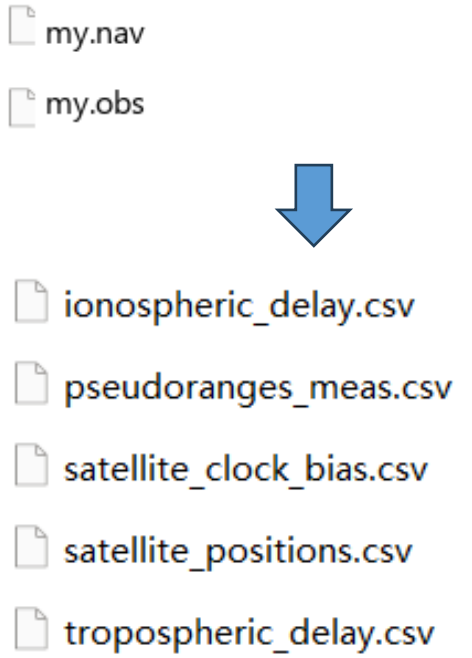
```
%% rinex2csv.m in the course repository (folder Lab1)
function rinex2csv(obs_file, nav_file, out_dir)
%   obs_file : RINEX observation file (.obs)
%   nav_file : RINEX navigation file (.nav)
%   out_dir  : output folder, created if needed
%   also writes epochs.csv and conversion_summary.txt
```

```
%% Convert RINEX to CSV (current folder: Lab1)
rinex2csv('mydata/my.obs', ...
          'mydata/my.nav', ...
          'outputs/own_inputs');
```

```
% rinex2csv.m adds matRTKLIB to the path itself and
% keeps GPS L1, elevation >= 15 deg, C/N0 >= 15 dB-Hz
```

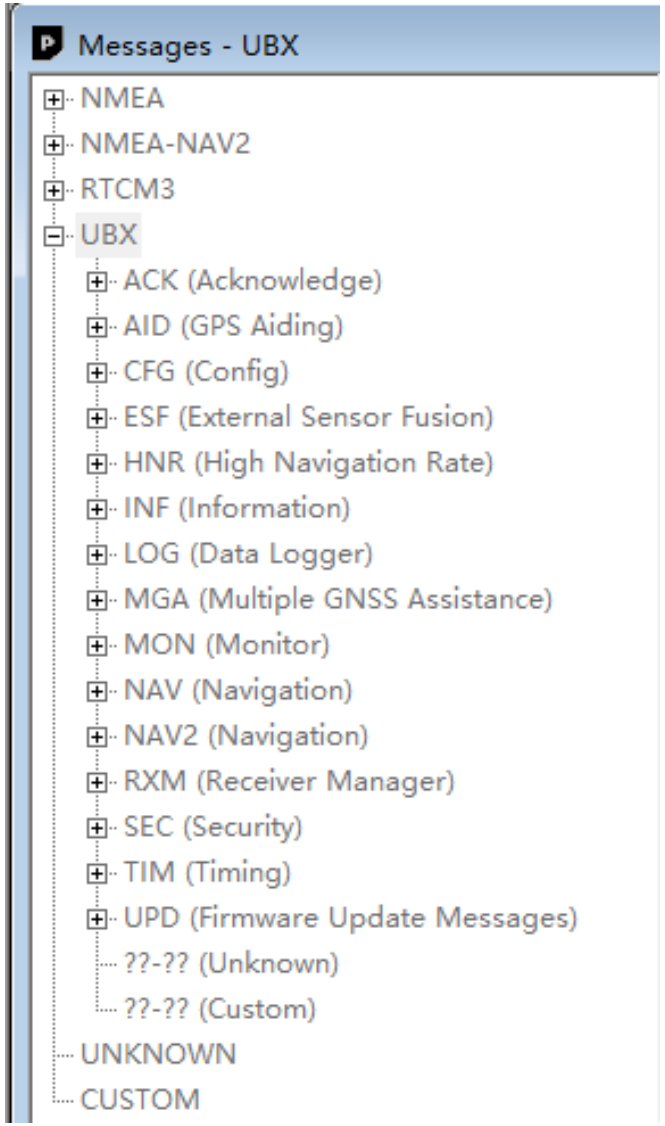
```
%% Optional: look at your data with MatRTKLIB
addpath('matRTKLIB');
obs = gt.Gobs('mydata/my.obs');
nav = gt.Gnav('mydata/my.nav');
obs.plotNSat;
obs.plotSky(nav);
```

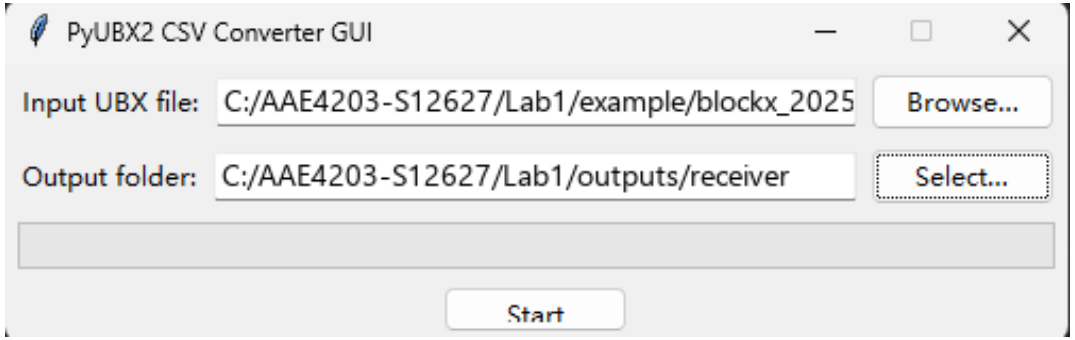
```
% Inside rinex2csv.m: one CSV file per quantity
writematrix(P, fullfile(out_dir, 'pseudoranges_meas.csv'));
writematrix(D, fullfile(out_dir, 'satellite_clock_bias.csv'));
writematrix(I, fullfile(out_dir, 'ionospheric_delay.csv'));
writematrix(T, fullfile(out_dir, 'tropospheric_delay.csv'));
writematrix(S, fullfile(out_dir, 'satellite_positions.csv'));
```



Step 8 · pyubx2: Receiver Solution to CSV

pyubx2_csv_converter_gui.py





2025-Sep-28_161220

ACK-ACK.csv
CFG-VALGET.csv
INF-DEBUG.csv
MON-COMMS.csv
MON-HW.csv
MON-HW2.csv
MON-HW3.csv
MON-IO.csv
MON-MSGPP.csv
MON-RF.csv
MON-RXBUF.csv
MON-SPAN.csv
MON-SYS.csv
MON-TXBUF.csv
MON-VER.csv
NAV-CLOCK.csv
NAV-COV.csv
NAV-DOP.csv
NAV-EOE.csv
NAV-GEOFENCE.csv
NAV-HPPOSECEF.csv
NAV-HPPOSLLH.csv
NAV-ODO.csv
NAV-ORB.csv
NAV-PL.csv
NAV-POSECEF.csv
NAV-POSLLH.csv
NAV-PVT.csv
NAV-RELPOSNED.csv
NAV-SAT.csv
NAV-SBAS.csv
NAV-SIG.csv
NAV-SLAS.csv
NAV-STATUS.csv
NAV-SVIN.csv
NAV-TIMEBDS.csv
NAV-TIMEGAL.csv
NAV-TIMEGLO.csv
NAV-TIMEGPS.csv
NAV-TIMELS.csv
NAV-TIMEQZSS.csv
NAV-TIMEUTC.csv
NAV-VELECEF.csv
NAV-VELNED.csv
RXM-MEASX.csv
RXM-RAWX.csv
RXM-RLM.csv
RXM-SFRBX.csv
SEC-SIG.csv
SEC-SIGLOG.csv
TIM-TP.csv

Receiver reference

Recap: Weighted Least-Squares SPP

What your Python code has to do at every epoch (Weeks 4–6)

$$\rho_i = \|\mathbf{p}^i - \mathbf{p}\| + b - c\,\delta t^i + I_i + T_i + \varepsilon_i$$

$$\mathbf{h}_i^T = [-\mathbf{e}_i^T \;\; 1], \qquad \Delta\mathbf{y} = \mathbf{H}\,\Delta\mathbf{x} + \boldsymbol{\varepsilon}$$

$$\Delta\hat{\mathbf{x}} = (\mathbf{H}^T\mathbf{W}\mathbf{H})^{-1}\mathbf{H}^T\mathbf{W}\,\Delta\mathbf{y}$$

ρ: pseudorange · p: receiver position · b: receiver clock bias (m) · δt^i: satellite clock · I, T: ionosphere, troposphere · e_i: unit line-of-sight vector · W: weights

Algorithm per epoch

1. Read pseudoranges and satellite positions
2. Correct satellite clock, ionosphere, troposphere
3. Start from x_0 (or the last solution)
4. Build Δy and H; solve for Δx
5. Update x; repeat until |Δx| < 1 mm
6. Write the ECEF position to the CSV

Typical bugs

cm vs m, ms vs s (NAV-HPPOSECEF)
wrong sign of the satellite clock term
satellites with missing corrections

Example inputs: the five CSV files produced by rinex2csv.m

Step 9 · Your Task: Least-Squares SPP in Python

Assignment: LSE_SPP Script (Python)

•**Goal**: Implement Single Point Positioning (SPP) using Least Squares Estimation (LSE).

•**Input Files**:

- `pseudoranges_meas.csv`
- `satellite_clock_bias.csv`
- `ionospheric_delay.csv`
- `tropospheric_delay.csv`
- `satellite_positions.csv`

•**Tasks**:

- Read and process the input data.
- Apply corrections (clock bias, ionosphere, troposphere).
- Estimate receiver position with LSE_SPP.
- Output estimated position (ECEF and/or LLH).
- Compare with the receiver solution from u-center (NAV-HPPOSECEF).

Validating Your Own Data Without Ground Truth

Your own data have no reference trajectory — so check them in three ways

Validation asks: is our processing chain working, and how does the solution behave? It does not give the true error.

1 Compare solutions

Your SPP vs RTKPOST SPP

Your SPP vs receiver solution (NAV-HPPOSECEF)

Differences in East, North, Up

2 Check consistency

Static antenna: the scatter is your noise

Mean, standard deviation, 95 % radius

Plot East vs North

3 Check the inputs

Satellites per epoch and per system

C/N_0 and elevation; DOP over time

Post-fit residuals: look for outliers

In the report: one table of mean and standard deviation of the E/N/U differences for the three solutions, and one East–North scatter plot.

Agreement is not accuracy: all three solutions can share the same multipath error

04

Benchmark Challenge & Leaderboard

Section 4 of 7

- The benchmark challenge
- How to improve accuracy
- Using AI tools
- Ranking, CSV format and upload

1 Lab Plan & Groups

2 Field Data Collection

3 Processing & Own-Data Validation

4 **Benchmark Challenge & Leaderboard**

5 Report & Checklist

6 Example Dataset: Block X

7 Resources

The Benchmark Challenge

Same algorithm, a harder dataset, an independent reference trajectory

The benchmark dataset

UrbanNav-HK-Deep-Urban-1, recorded outdoors in Hong Kong

RINEX observation + navigation files (u-blox)

Reference trajectory from a high-grade system

Harder than Block X: multipath and blocked signals

Your task

Estimate a position for every epoch

Improve your SPP one step at a time

Use any GNSS technique — only the observation and navigation files as input

Upload GXX_benchmark.csv — as often as you like

Baseline SPP → **One change** → **Check on example** → **Upload CSV** → **See your rank**

Keep a log of every version and its result — it becomes the ablation table of your report

How to Improve Accuracy

Change one thing at a time and measure its effect

Technique	Idea	Expected effect
Multi-constellation	add BDS and Galileo, one clock bias per system	more satellites, better geometry
Weighting	$\sin^2$(elevation) or C/N_0-based variances	down-weights weak, low signals
Masks	elevation 10–15° , C/N_0 25–30 dB-Hz	removes the worst signals
Outlier rejection	drop the largest normalised residual, re-solve	removes multipath / NLOS outliers
Atmosphere	check Klobuchar and Saastamoinen corrections	removes metre-level biases
Filtering	carrier smoothing or a Kalman filter with Doppler	reduces noise

$$\mathbf{x} = [x,\ y,\ z,\ b_G,\ b_C,\ b_E]^T, \qquad \mathbf{h}_i^T = [-\mathbf{e}_i^T\ \ 0\ \ 1\ \ 0]\ \ \text{(BDS satellite)}$$

Multi-constellation usually gives the largest single gain in Hong Kong

Using AI Tools: Allowed, with Rules

ChatGPT, Claude, Gemini, DeepSeek, Copilot … may be used to write and debug code

Good uses	Rules
Generate a first WLS-SPP from the CSV inputs	Every member can explain the algorithm and the code
Extend it: GPS + BDS + Galileo, weighting, outlier rejection	Verify each version on the example dataset first
Explain error messages and plots	Report: AI-use statement + key prompts in an appendix
Write the evaluation script for the example dataset	The CSV must be computed by your code from the provided observations

Example prompt

"My inputs are pseudoranges_meas.csv, satellite_positions.csv, satellite_clock_bias.csv, ionospheric_delay.csv and tropospheric_delay.csv (columns: …). Write a Python function for weighted least-squares SPP per epoch that returns ECEF positions, and explain each step."

AI writes code fast; you are responsible for checking that it is right

How You Are Ranked

Benchmark only · errors in the local East-North-Up frame at the reference position

$$\mathrm{RMSE}_{2D} = \sqrt{\frac{1}{N}\sum_{k=1}^{N}\left(e_{E,k}^2 + e_{N,k}^2\right)}, \qquad A = 100\,\frac{N}{N_{ref}}\ \%$$

Ranking rules

Ranked only if availability A ≥ 90 %

Smallest 2D RMSE first; ties broken by 3D RMSE

95th-percentile 2D error also shown

Fair play

Your GitHub code must reproduce the CSV

The teaching team may re-run any group's code

Weight of the rank in the lab mark: on Blackboard

Using the public UrbanNav ground truth = disqualification

Epochs matched by GPS week + time of week · missing epochs lower the availability

Submission File Format (CSV)

One header line + one row per benchmark epoch, in time order

Column	Meaning	Unit / format
gps_week	GPS week number	integer
gps_tow_s	GPS time of week of the epoch	s, at least 1 decimal
x_ecef_m	Estimated receiver position, ECEF X	m, 3 decimals
y_ecef_m	Estimated receiver position, ECEF Y	m, 3 decimals
z_ecef_m	Estimated receiver position, ECEF Z	m, 3 decimals

File names

GXX_benchmark.csv
GXX_owndata.csv
e.g. G05_benchmark.csv

```
gps_week,gps_tow_s,x_ecef_m,y_ecef_m,z_ecef_m
2158,455346.0,-2418202.586,5385847.524,2405648.799
2158,455347.0,-2418202.174,5385847.238,2405648.952
…
```

ECEF in metres (WGS-84) · no extra columns · run python tools/check_submission.py before uploading

Upload and Leaderboard

Google Drive folder from the TA (link on Blackboard), one sub-folder per group

Compute CSV → Run the checker → Upload to Drive → Auto-scoring → Leaderboard

During the session	After the session
Upload as soon as you have a first solution	Keep improving until the deadline (on Blackboard)
Replace the file whenever you improve it	The file present at the deadline is final
Live leaderboard link from TA; preliminary ranking at about 2:50	Final leaderboard on Blackboard; the reference trajectory is not released

Path: TA's Drive folder / G05 / G05_benchmark.csv (use your own group number; put GXX_owndata.csv in the same sub-folder)

Only the latest file named GXX_benchmark.csv is evaluated

What the Leaderboard Looks Like

Illustrative numbers only — the real leaderboard is shown during the session

Rank	Group	Version	2D RMSE (m)	3D RMSE (m)	2D P95 (m)	Availability
1	G07	GPS+BDS+GAL, C/N_0 weights, outlier rejection	6.8	14.9	12.1	99.4 %
2	G15	GPS+BDS, elevation weights	9.3	18.2	17.5	100 %
3	G02	GPS+BDS, elevation mask 15°	12.6	24.0	23.8	97.1 %
…	…	…	…	…	…	…
18	G11	GPS only (baseline)	31.4	45.7	52.0	100 %
–	G04	GPS only, many epochs missing	8.9	16.3	15.0	71.0 % < 90 %

The last row is not ranked: a low error on a few easy epochs does not count when availability is below 90 %.

Version column: optional GXX_version.txt in your group's Drive folder

05

Report & Checklist

Section 5 of 7

- Lab report requirements
- Checklist

1 Lab Plan & Groups

2 Field Data Collection

3 Processing & Own-Data Validation

4 Benchmark Challenge & Leaderboard

5 **Report & Checklist**

6 Example Dataset: Block X

7 Resources

Lab Report Requirements

One report per group · ≤ 10 pages excluding the cover · code listings ≤ 2 pages

Part	Required content
1. Data collection	set-up, site photos, time, environment near Block X
2. Raw data analysis	satellites per constellation, C/N_0, sky plot, DOP
3. SPP algorithm	model, linearisation, weighted least squares, your code
4. Own-data validation	your SPP vs RTKLIB and the receiver solution; scatter of the static solution
5. Benchmark results	each technique tried, its measured effect, final metrics and rank
6. AI-use statement	tools, purposes, key prompts (appendix), how you verified the output

Template on Blackboard (PolyU_AAE4203_Lab_report_template.docx) · GitHub repository with raw data, code and README · deadline announced on Blackboard

Your repository must reproduce both CSV files

Checklist

Tick everything before you leave the field, the lab, and before the deadline

Before leaving the field

- ☐ UBX file saved and copied to a second place
- ☐ Photos of the set-up and the sky
- ☐ Start/end time, weather
- ☐ Antenna position marked on a map
- ☐ Kit complete, handed over

Before the session ends

- ☐ Example results reproduced
- ☐ Own data converted and validated
- ☐ Validation plots saved
- ☐ Version log started
- ☐ Benchmark CSV checked and uploaded

Before the deadline

- ☐ Final CSV files uploaded
- ☐ Report (PDF) submitted
- ☐ Repository reproduces both CSV files
- ☐ AI-use statement and prompts included
- ☐ Every member can explain the code

No data, no report: back up the raw UBX file before leaving the site

06

Example Dataset: Block X

Section 6 of 7

- What it contains and expected results
- Site, set-up and RTKLIB
- Python WLS-SPP results

1 Lab Plan & Groups
2 Field Data Collection
3 Processing & Own-Data Validation
4 Benchmark Challenge & Leaderboard
5 Report & Checklist
6 **Example Dataset: Block X**
7 Resources

The Example Dataset: Your Warm-Up

Recorded at Block X in 2025 · the following slides walk through it step by step

What you get

Raw .ubx log, RINEX .obs/.nav, HK SatRef base .obs, the five CSV files

Receiver solution (NAV-HPPOSECEF.csv)

Example Python WLS-SPP code and its plots

How to use it

Run python WLSE_spp.py before the lab; compare with the table

Re-run after every change to your code

Different numbers? Check units, time and file paths

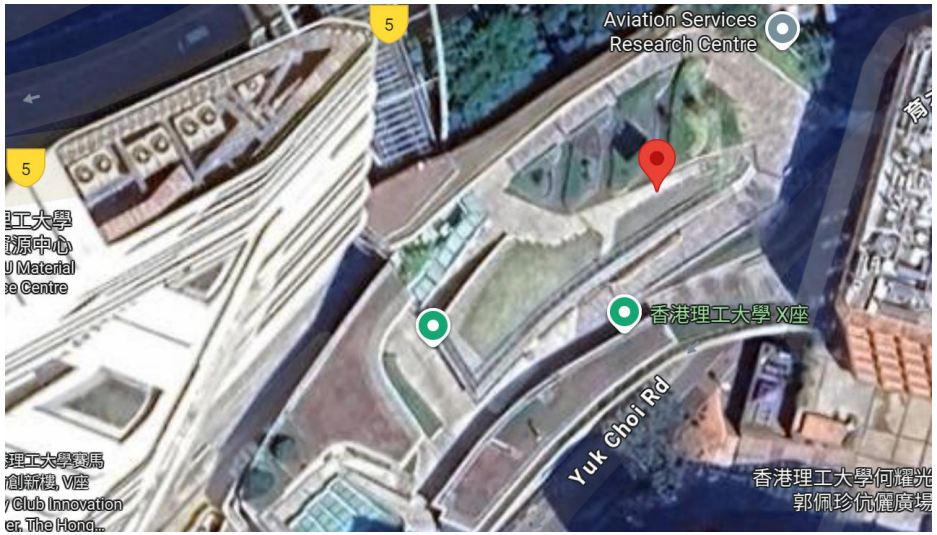
Expected result (GPS only)	2D	3D
Epochs	186	186
Mean error (m)	37.50	49.56
RMS error (m)	37.91	51.04
95th percentile (m)	45.21	73.72
Maximum (m)	51.70	113.73

Errors relative to the receiver solution. In the 2025 plots the receiver solution is labelled "Ground Truth".

If you reproduce these numbers, your environment and code are working

Example: Data Collection Site (Block X)

Experimental Site



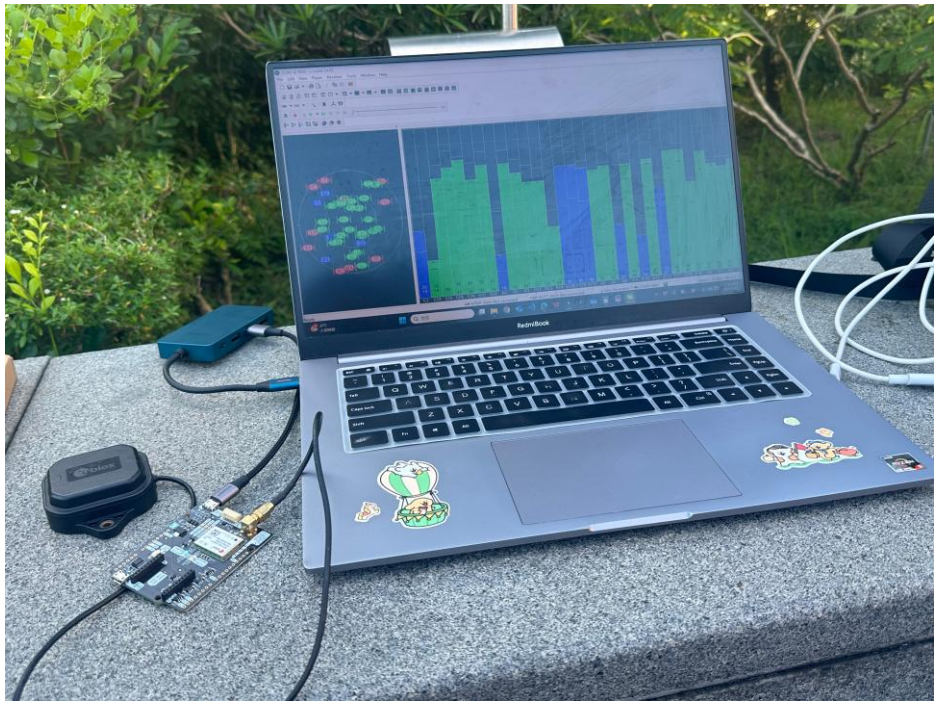
The experiment is to be geographically **restricted to Block X, The Hong Kong Polytechnic University**

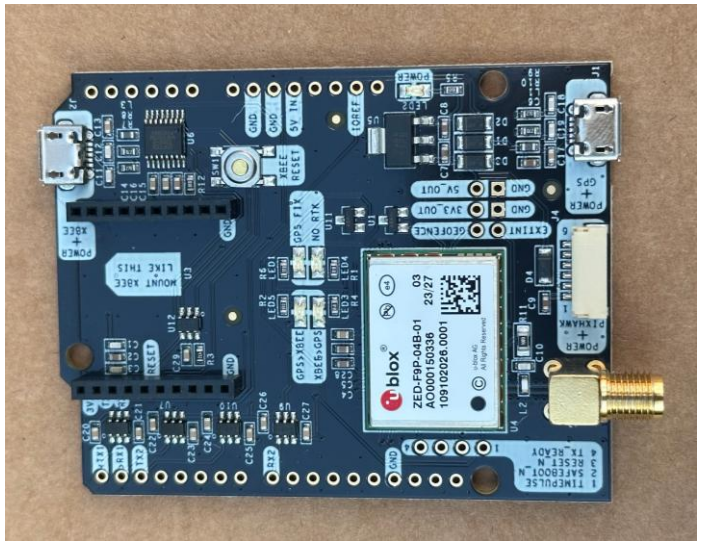
Experimental Environment



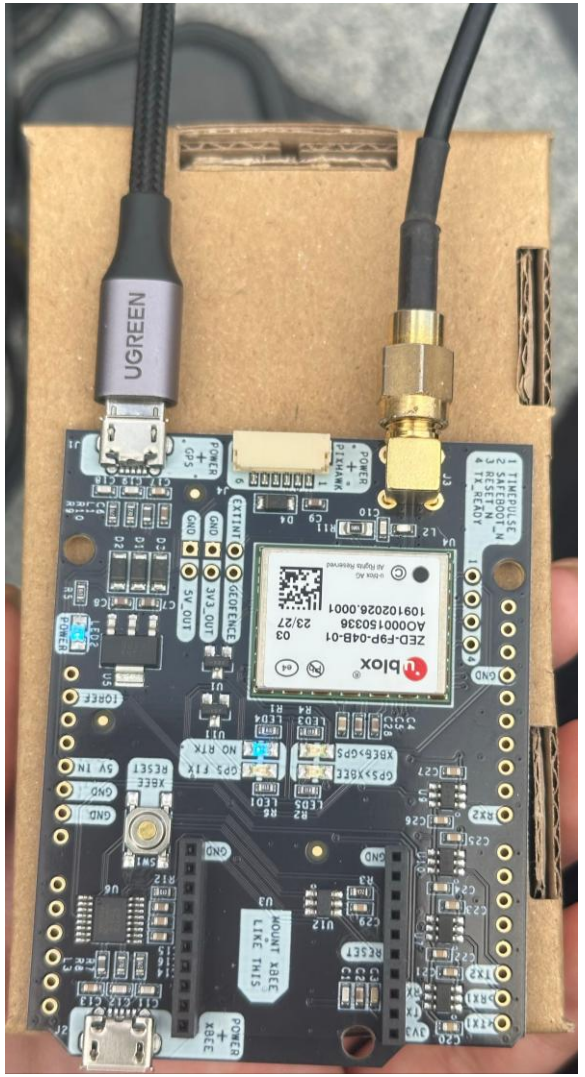


Example: Equipment Set-up









The equipment used for data collection:

- One laptop
- U-blox ZED-F9P + Antenna / U-blox boards

Device connection details

Example: Data Collection



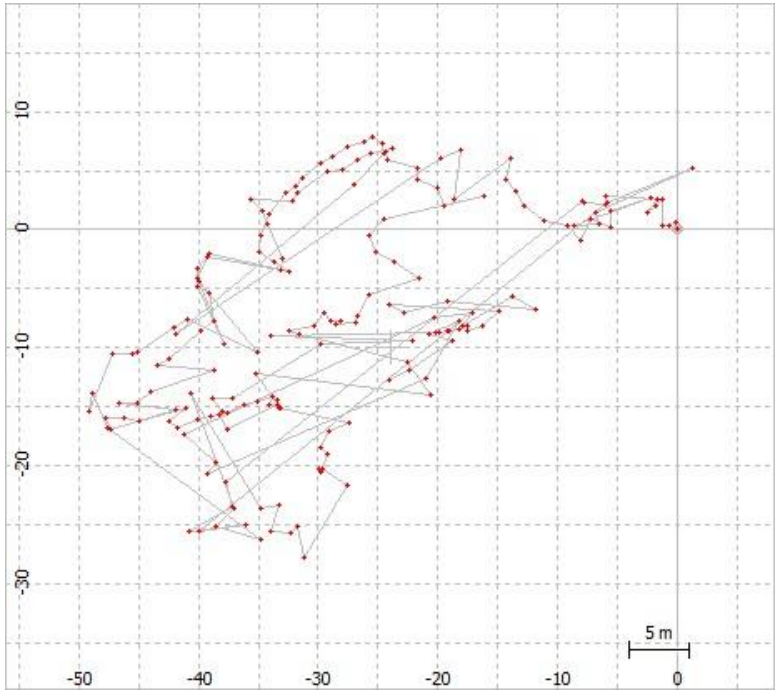
Example: RTKLIB Processing

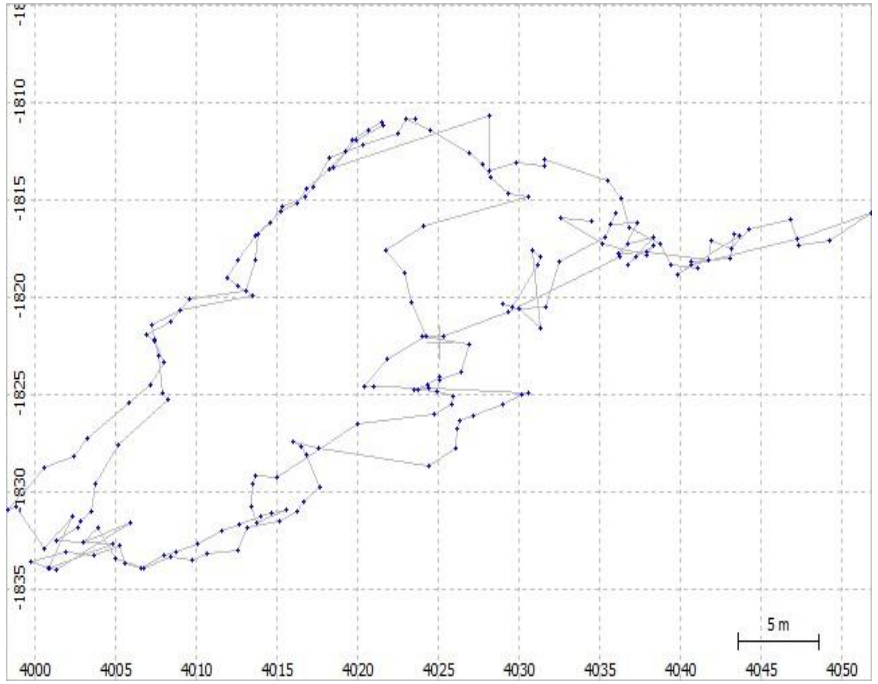
blockx_2025.ubx

blockx_2025.obs
blockx_2025.nav

RTKCONV







Receiver reference

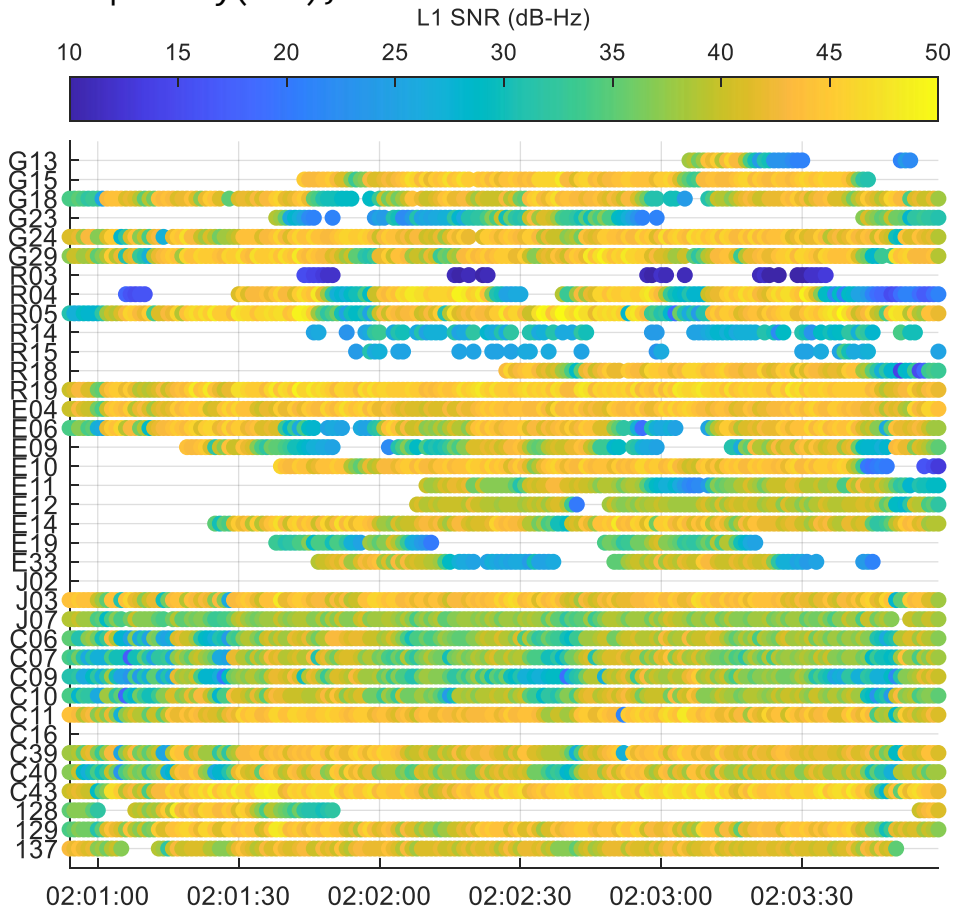
RTKPOST:
Single Point Positioning
(SPP)

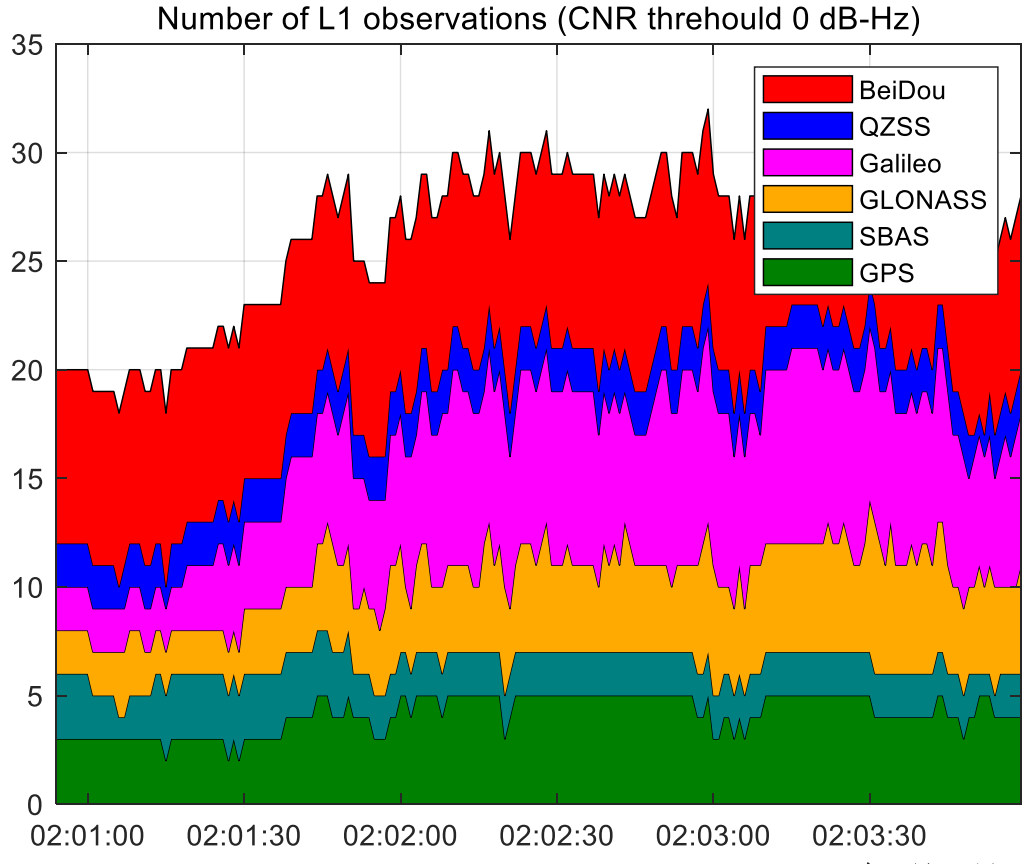
RTKPOST:
DGNSS Positioning

Example: matRTKLIB rinex2csv (1/2)

1. matRTKLIB

```
%% Plot the example with MatRTKLIB (current folder: Lab1)
addpath('matRTKLIB');
obs = gt.Gobs('example/blockx_2025.obs');
nav = gt.Gnav('example/blockx_2025.nav');
obs.plot;
obs.plotNSat;
obs.plotSky(nav);
```



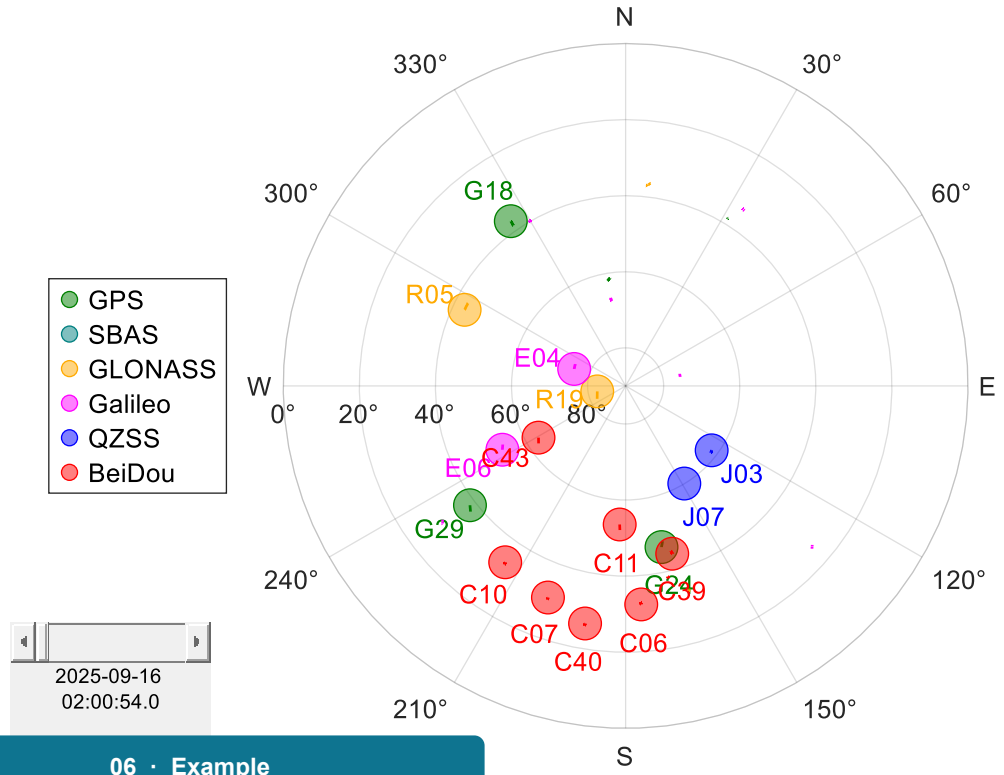


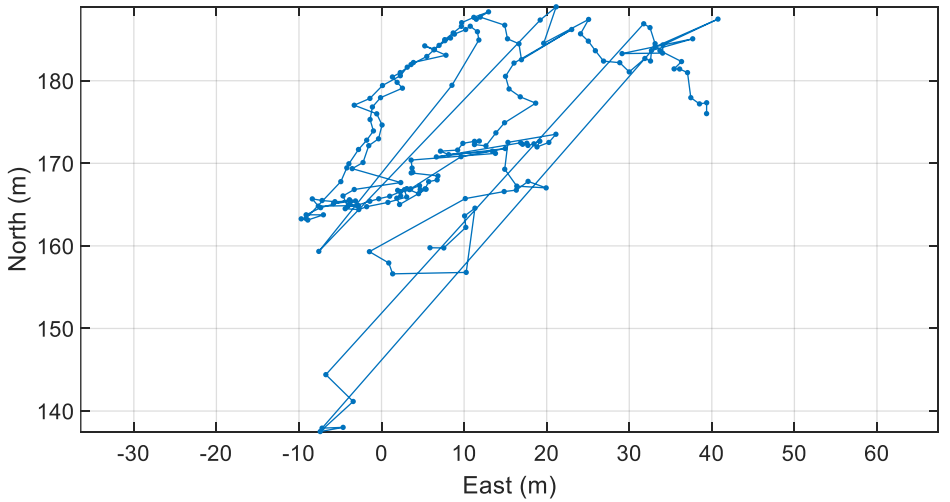
Example: matRTKLIB rinex2csv (2/2)

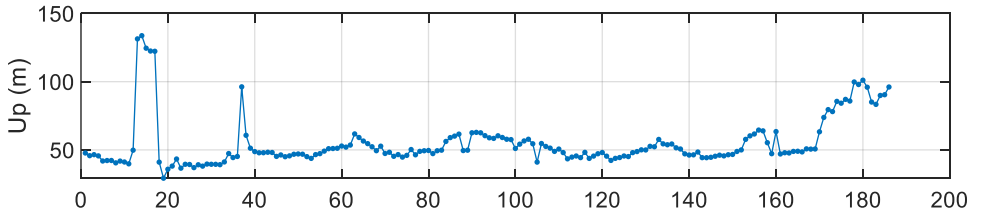
1. matRTKLIB

```
%% Plot the example with MatRTKLIB (current folder: Lab1)
addpath('matRTKLIB');
obs = gt.Gobs('example/blockx_2025.obs');
nav = gt.Gnav('example/blockx_2025.nav');
obs.plot;
obs.plotNSat;
obs.plotSky(nav);
```

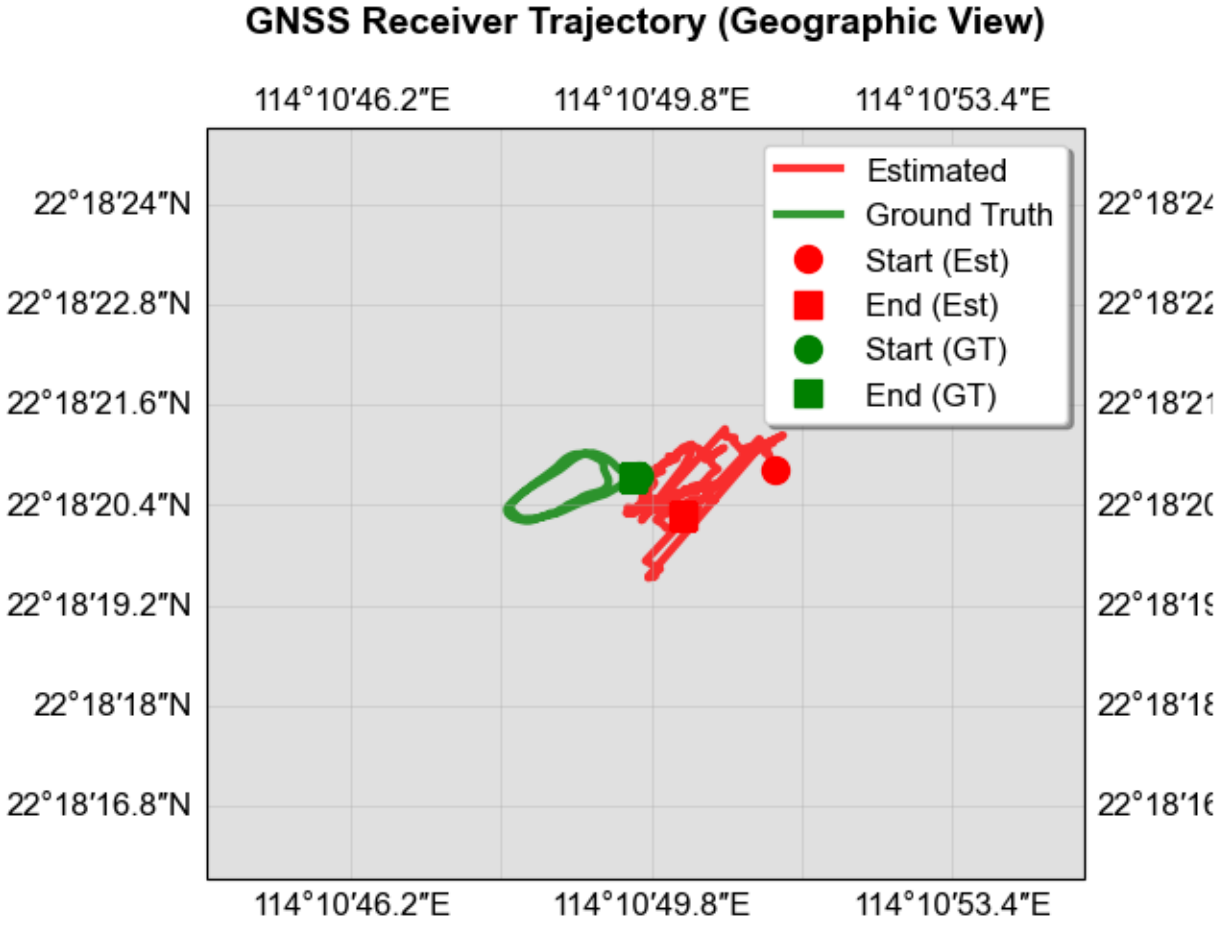
```
% Convert the example to CSV (current folder: Lab1)
rinex2csv('example/blockx_2025.obs', ...
          'example/blockx_2025.nav', ...
          'outputs/example_inputs');
% The next slides use the five CSV files provided in example/:
% python WLSE_spp.py --reference example/NAV-HPPOSECEF.csv
```

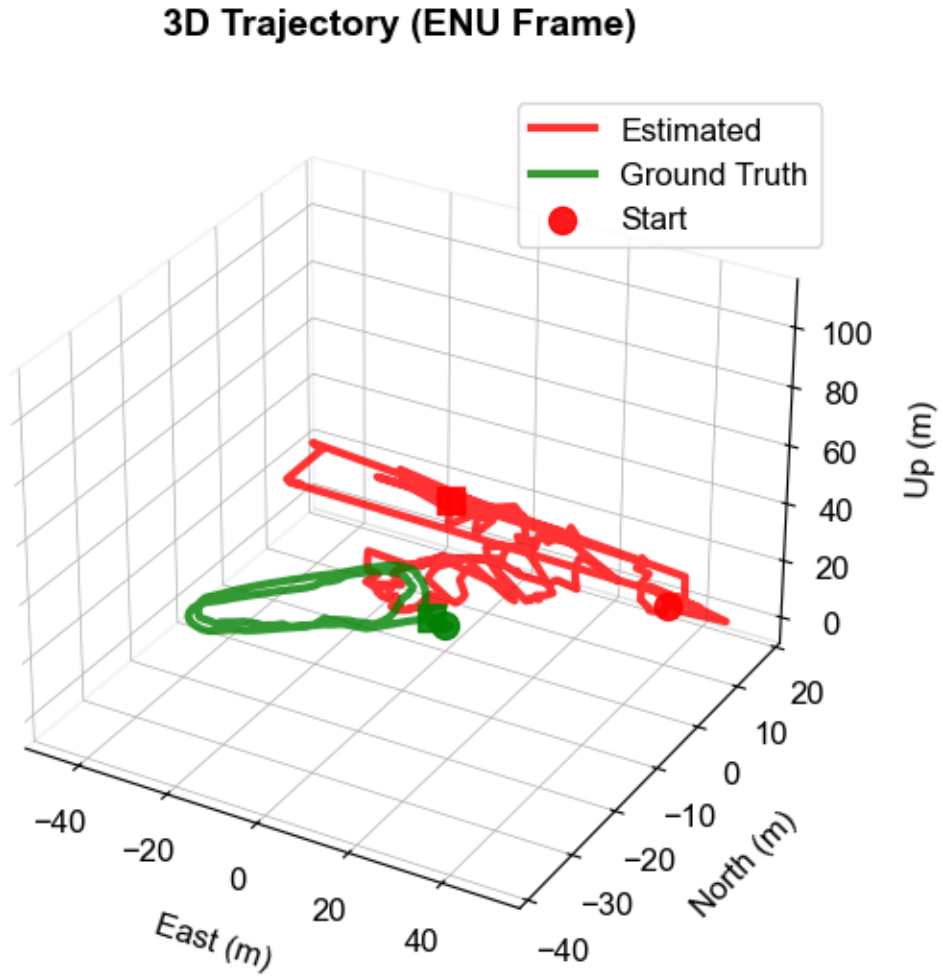




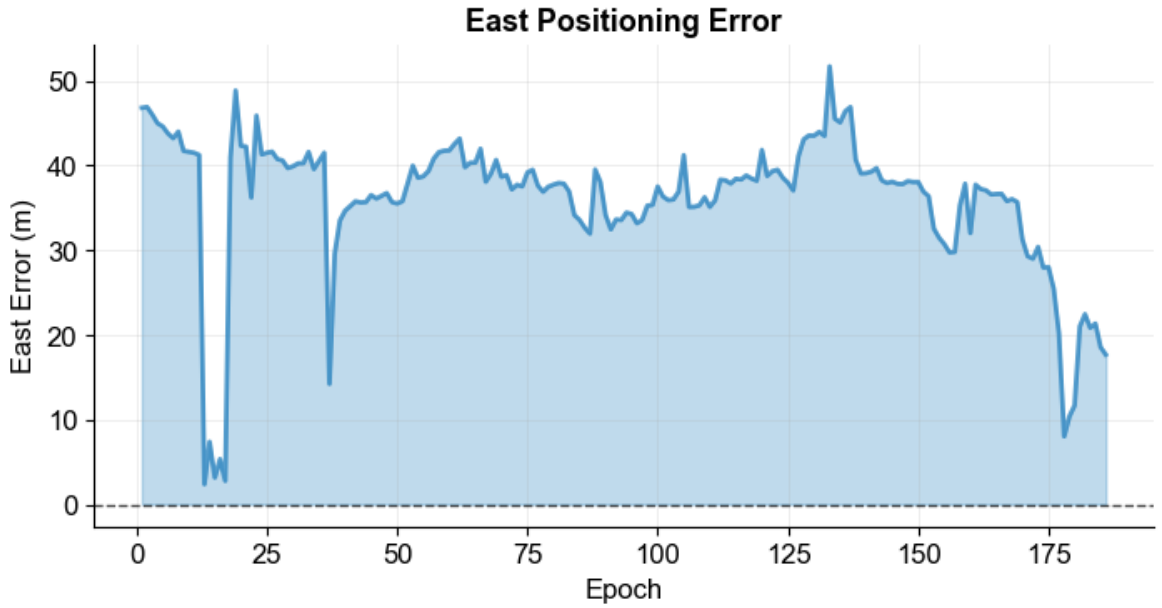


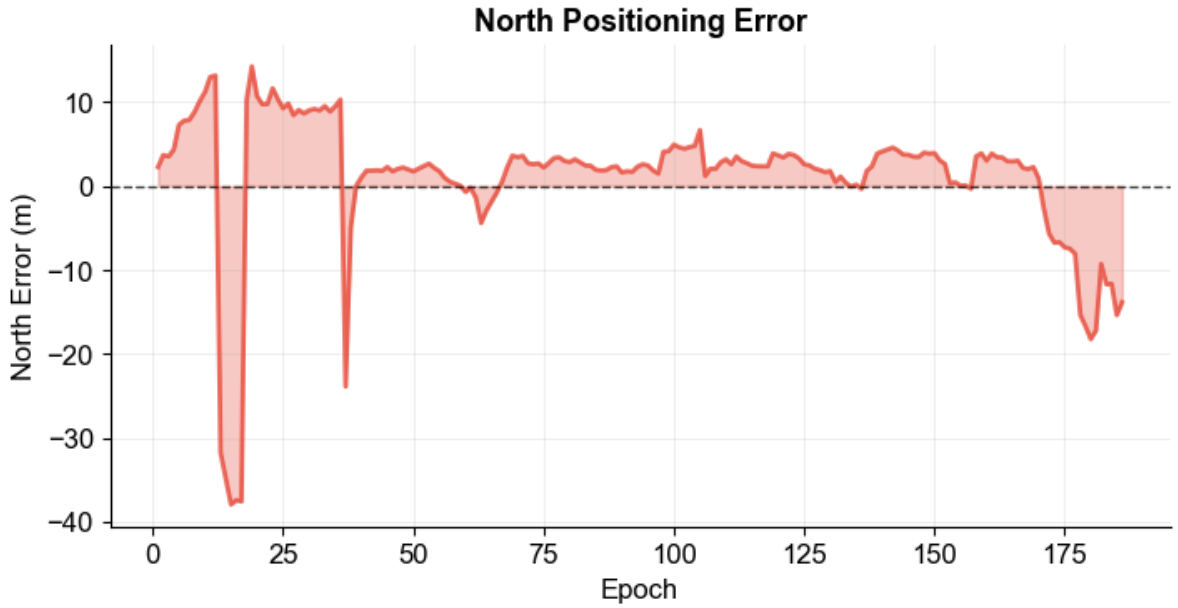
Example: Python WLSE-SPP Results (1/5)

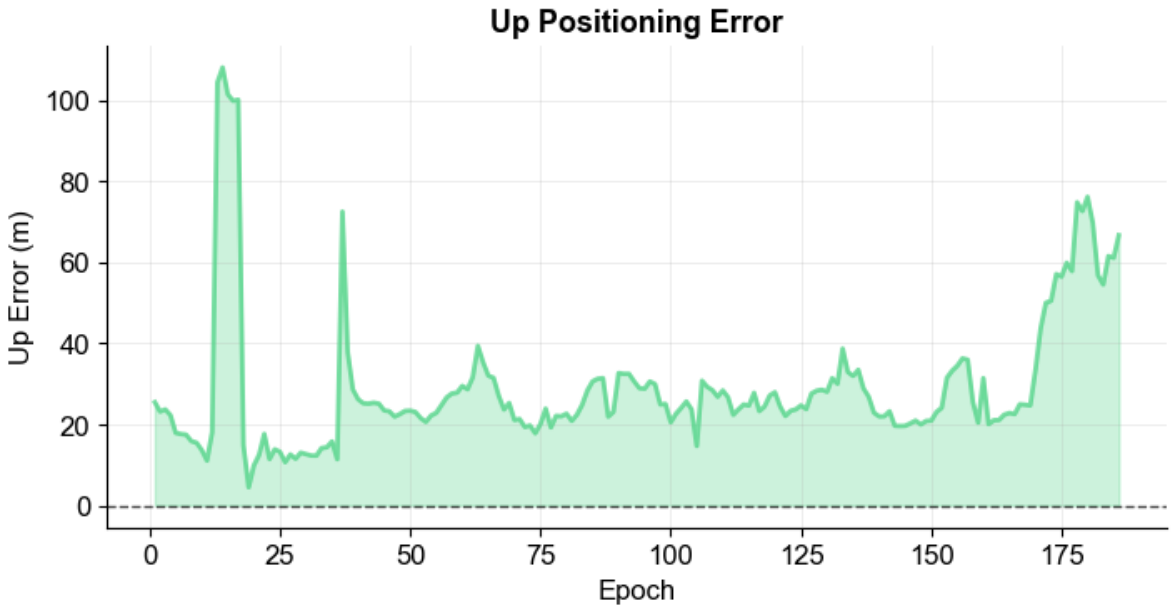


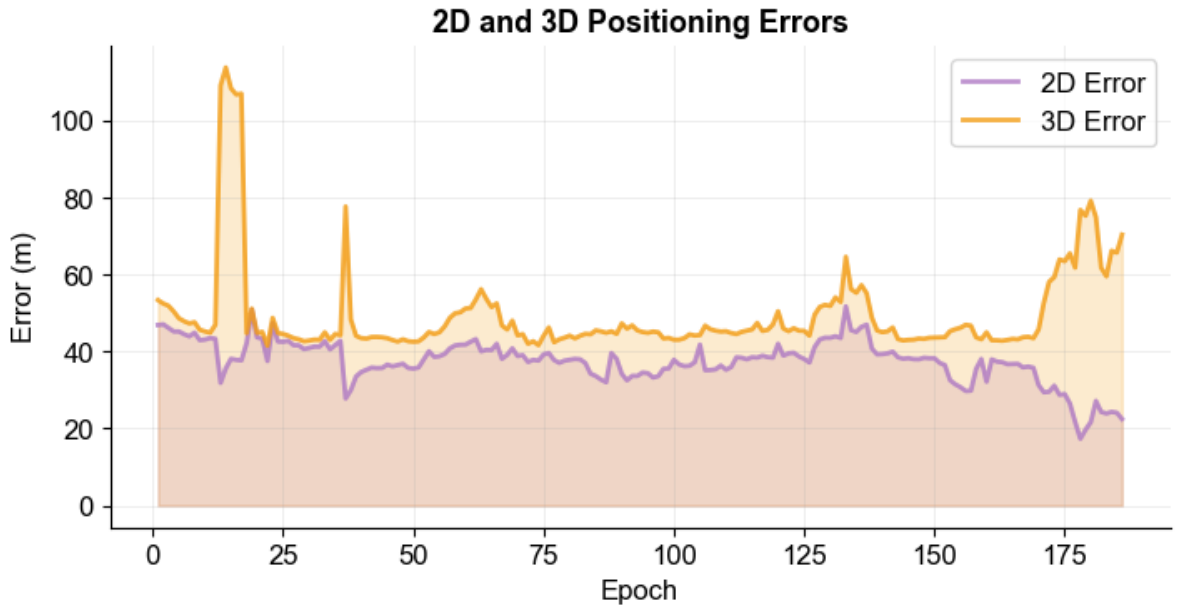


Example: Python WLSE-SPP Results (2/5)

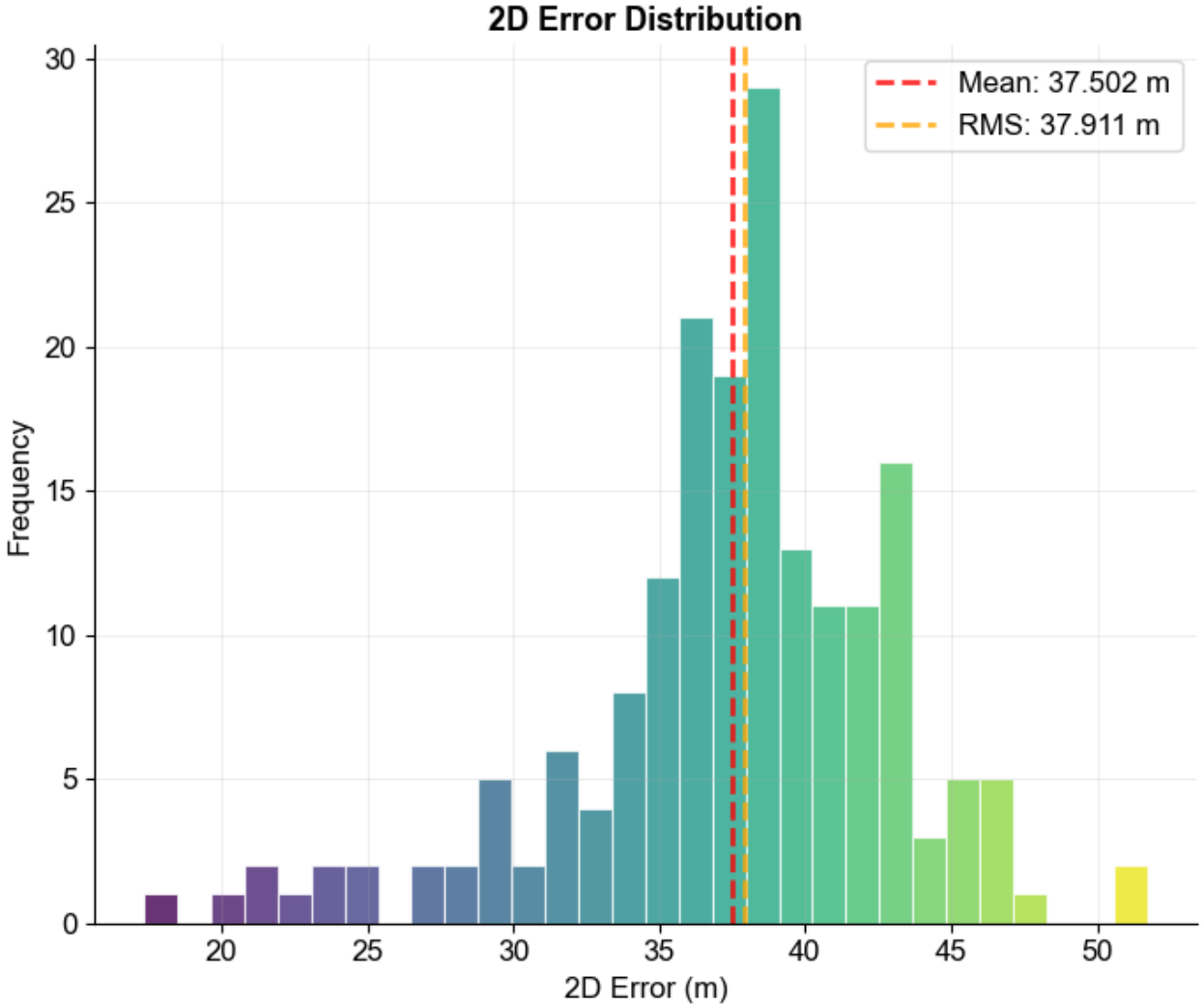


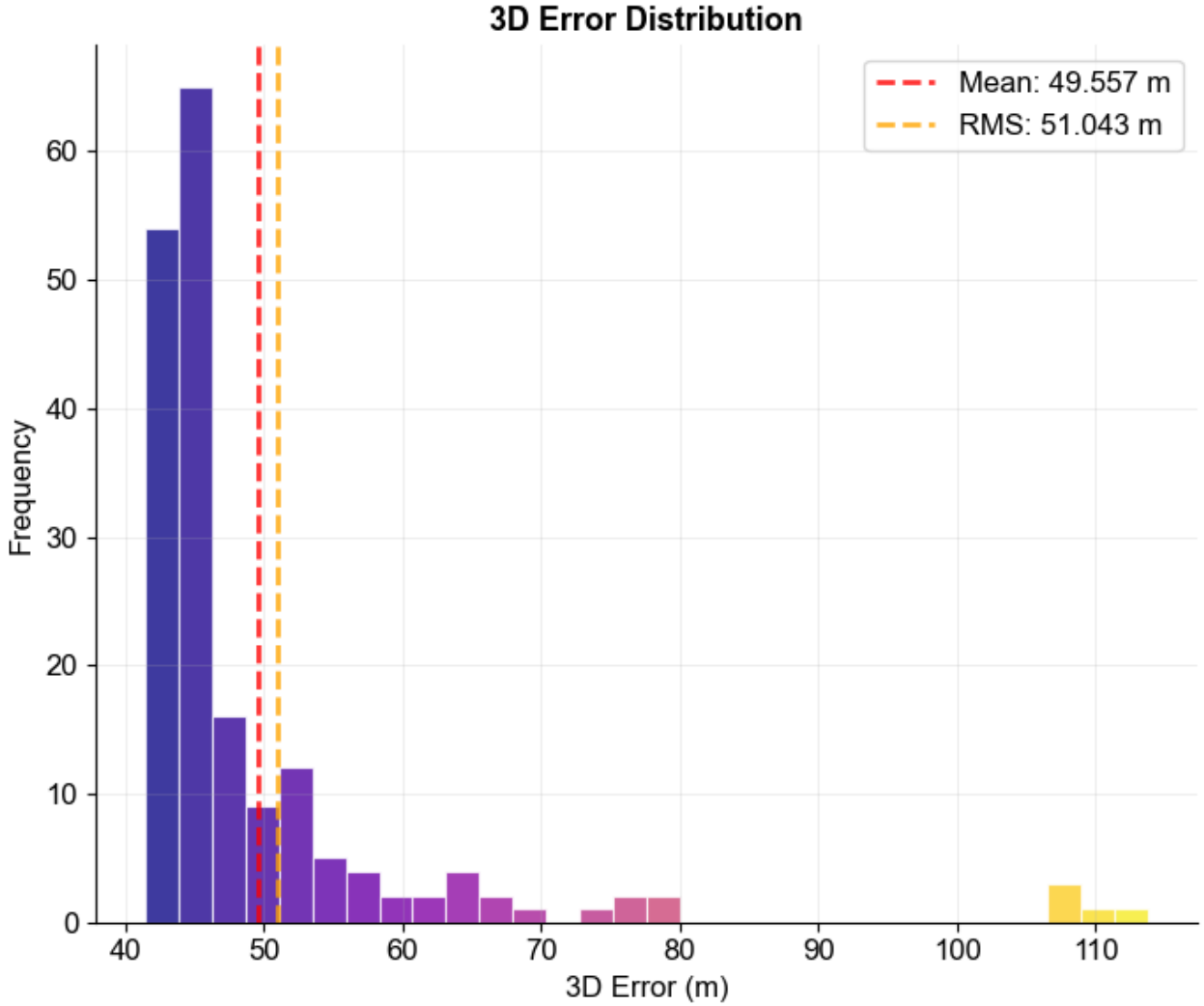




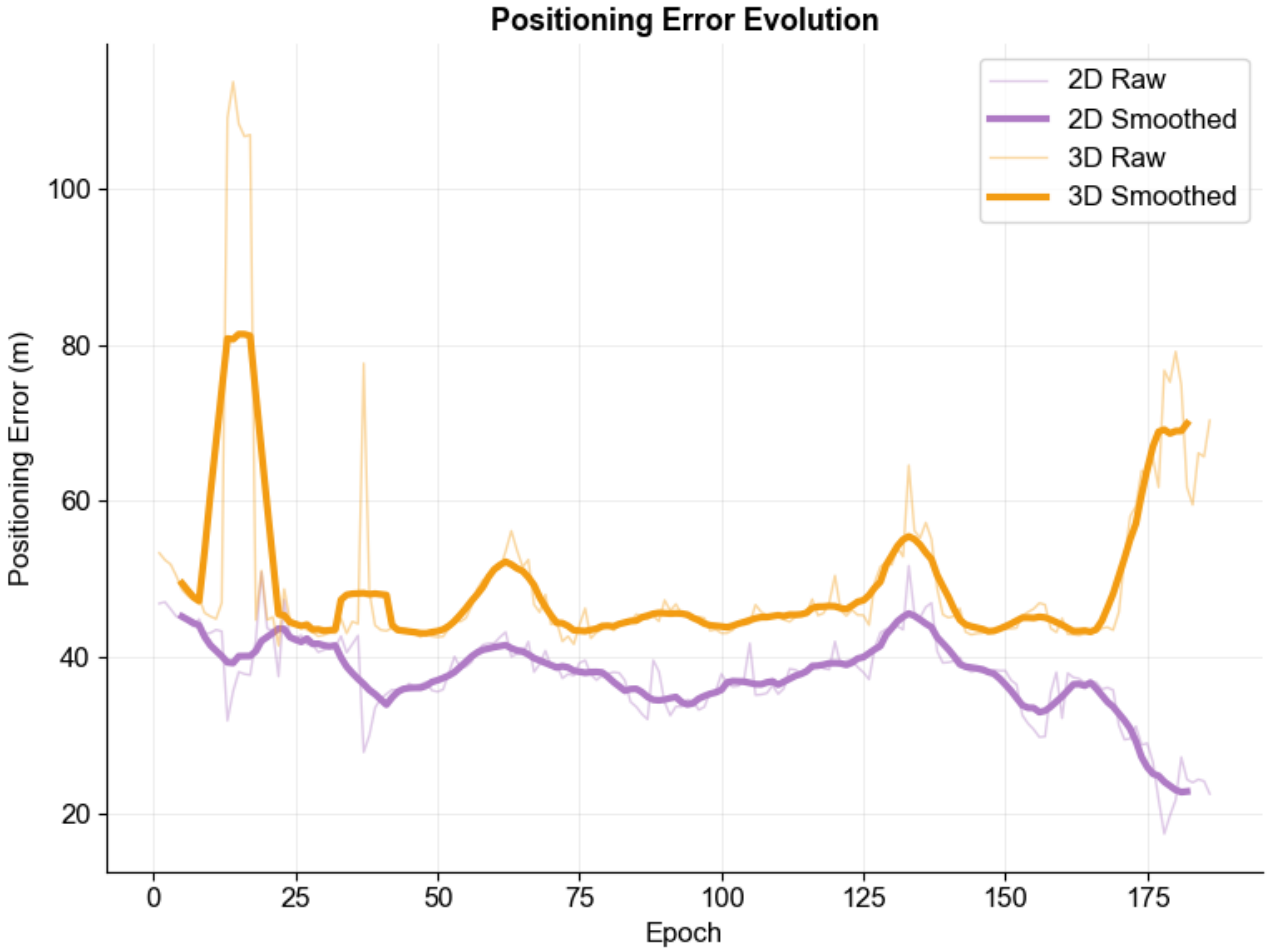


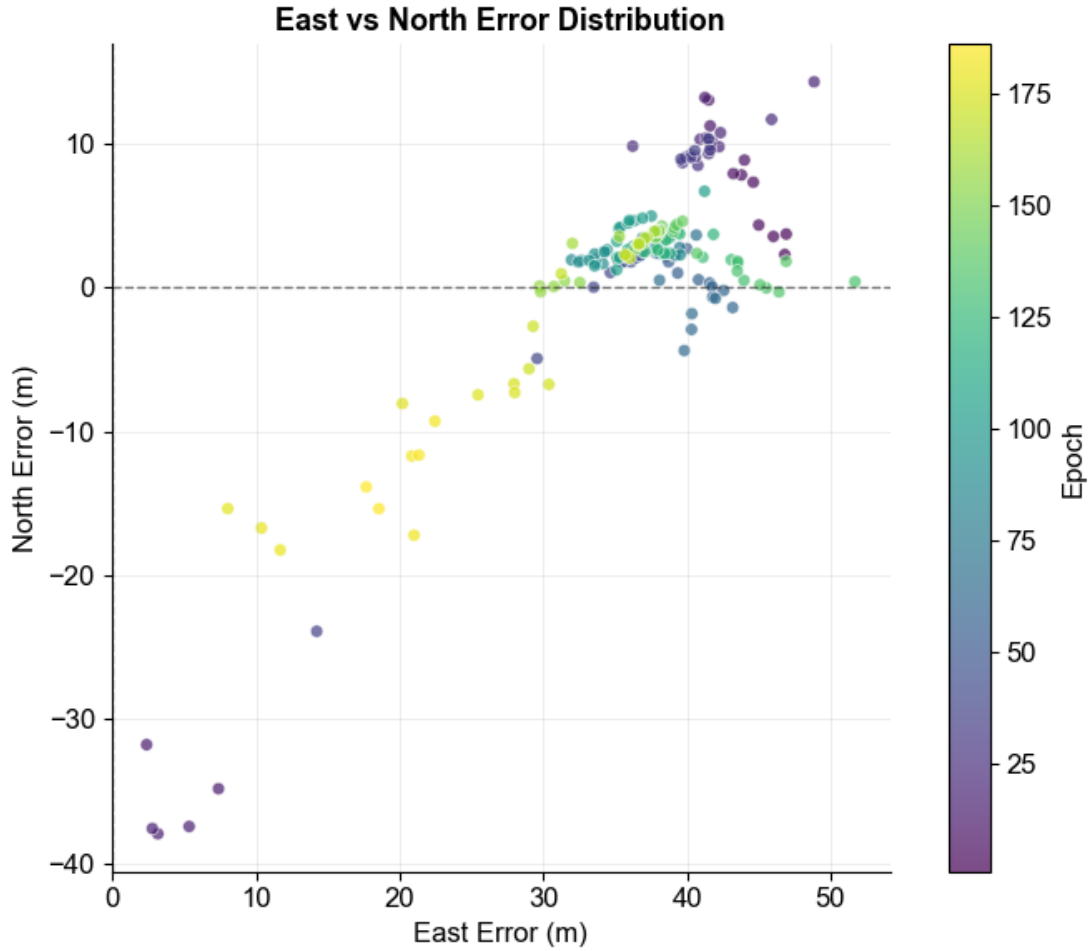
Example: Python WLSE-SPP Results (3/5)





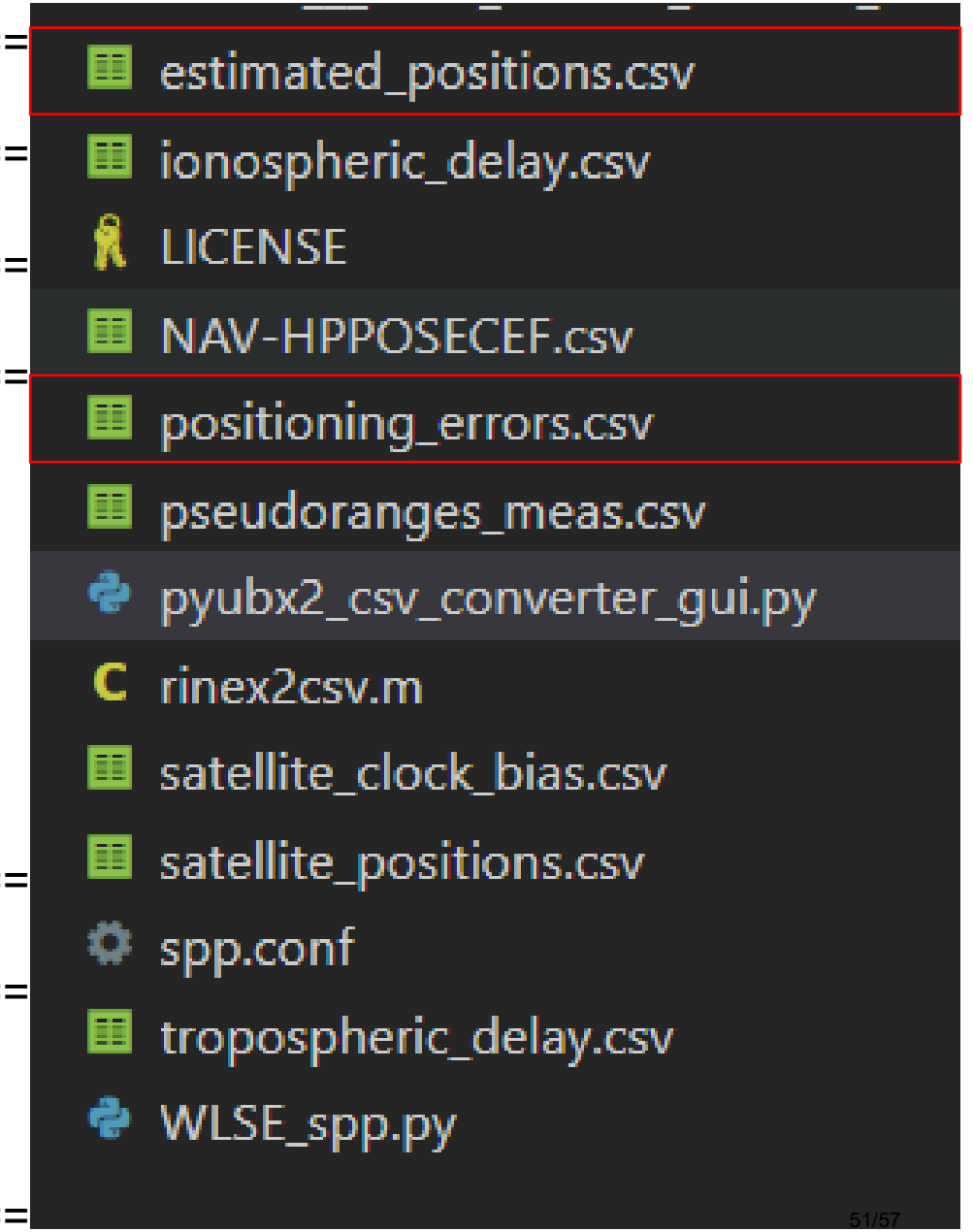
Example: Python WLSE-SPP Results (4/5)





Example: Python WLSE-SPP Results (5/5)

```
================================================================
        GNSS SPP POSITIONING RESULTS SUMMARY
================================================================
Total Epochs Processed: 186
================================================================
        POSITIONING ACCURACY STATISTICS
================================================================
Metric              2D Error (m)    3D Error (m)
----------------------------------------------------
Mean                 37.502          49.557
RMS                  37.911          51.043
Standard Deviation    5.553          12.227
Maximum              51.699         113.729
Minimum              17.350          41.472
95th Percentile      45.208          73.716
================================================================
              COORDINATE RANGES
================================================================
Latitude Range:  22.305424°  to 22.305920°
Longitude Range: 114.180416°  to 114.180938°
Altitude Range:  10.550 m to 114.610 m
================================================================
```



07

Resources

Section 7 of 7

- Lab tutorial
- Software documentation
- Further reading

1 Lab Plan & Groups
2 Field Data Collection
3 Processing & Own-Data Validation
4 Benchmark Challenge & Leaderboard
5 Report & Checklist
6 Example Dataset: Block X
7 **Resources**

Resources and References

[1] AAE4203 Laboratory Tutorial 2026/27 (Word/PDF) — full instructions, theory and worked examples; on Blackboard.

[2] T. Takasu, RTKLIB Manual v2.4.2; rtklibexplorer RTKLIB 2.5 (RTKCONV, RTKPOST, RTKPLOT).

[3] u-blox, u-center User Guide; ZED-F9P Integration Manual (UBX protocol).

[4] L.-T. Hsu, N. Kubo, W. Wen et al., “UrbanNav: an open-sourced multisensory dataset for benchmarking positioning algorithms designed for urban areas,” ION GNSS+ 2021 — the source of the benchmark dataset.

[5] pyubx2 (UBX parsing) and MatRTKLIB (rinex2csv.m) — links in the tutorial.

[6] Lands Department, HKSAR, SatRef RINEX data, www.geodetic.gov.hk.

[7] P. Misra and P. Enge, Global Positioning System: Signals, Measurements, and Performance, 2nd ed., 2011.

[8] AAE4203 course GitHub repository 2026/27 (code and tools): github.com/weisongwen/AAE4203-S12627 (folder Lab1)

Thank You

Dr Weisong Wen · Assistant Professor
Trustworthy AI and Autonomous Systems Laboratory (TAS Lab)
Department of Aeronautical and Aviation Engineering
The Hong Kong Polytechnic University
Email: welson.wen@polyu.edu.hk · Office: R820 · Phone: 3400 8234

Questions?

If you have any questions, please contact me or the TAs.



TAS Lab, 2026

A

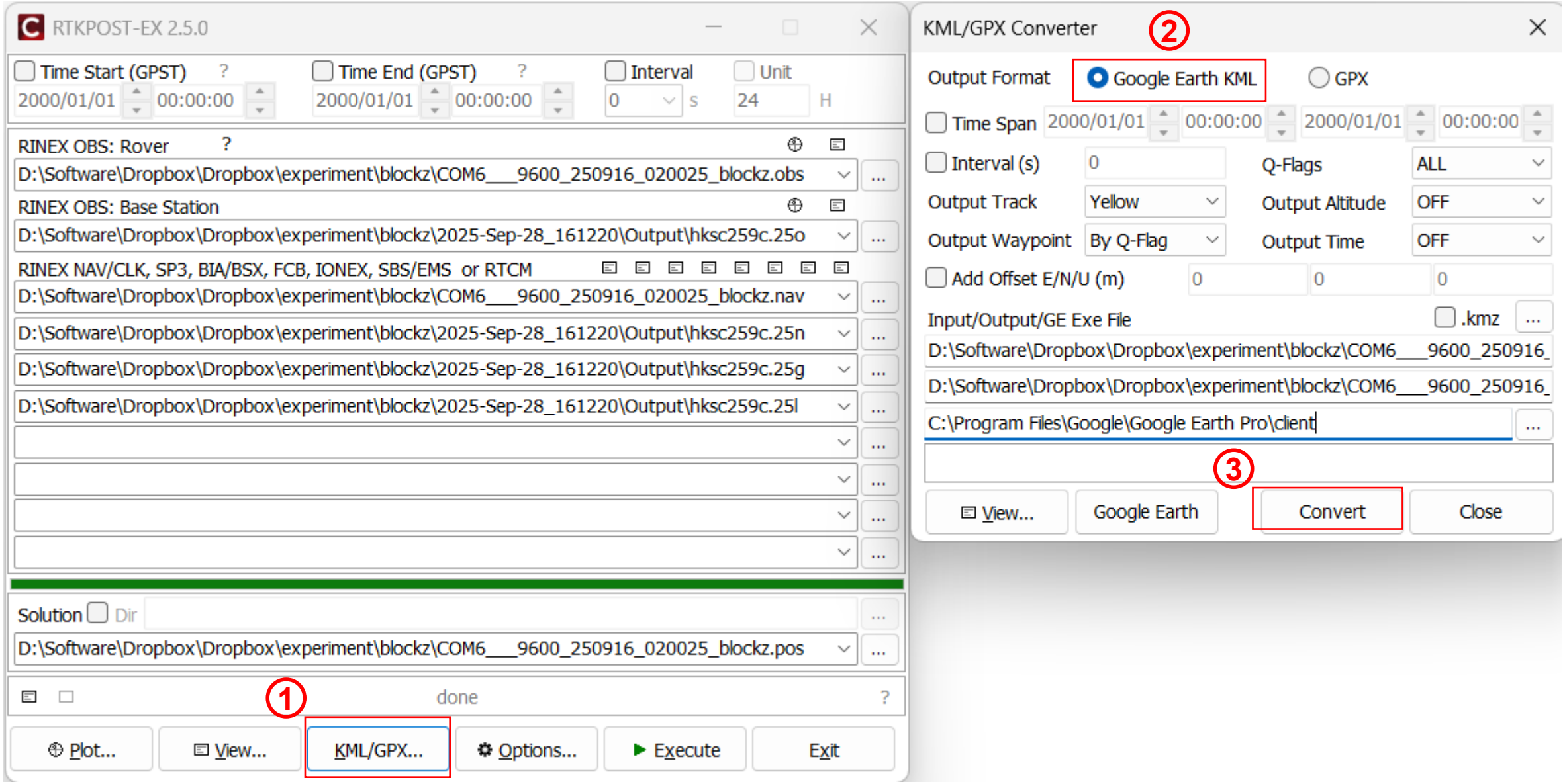
Additional RTKLIB Screenshots

Supplementary material

- RTKPOST settings
- RTK vs ground truth

A **Appendix A: Additional RTKLIB Screenshots**

Appendix: RTKPOST Settings (Reference)



Appendix: RTK Solution vs Receiver Reference

